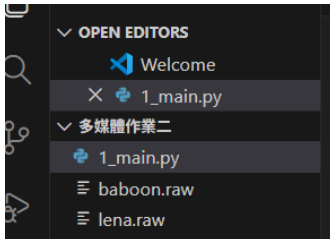


多媒體作業_HW02

- 工作進度編排:
 - 作業題目一_Step 1:先做老師給的兩個 .raw 檔案做影像存取:
影像大小是 512*512, 然後每個 pixel 使用的 RGB 會各是 8-bit (24-bit 彩色), 然後因為是 raw格式所以沒有檔頭, 會是把所有像素質一個個存起來, 排列大小是 sequential 次序(會先存完整整張圖的 R 平面, 再存 G, 再存 B)。
 - 實作方法:因為 RAW 沒有額外的資訊, 像是寬、高、格式, 所以程式必須自己知道寬高與排列的方式, 才能把一維的 byte 串還原成三個 2D 的 R、G、B 影像;然後我是使用 vs code (用 python 撰寫)。
- (1) 建立資料夾:這邊先將老師給的兩個 .raw 檔案拉進去資料夾, 再建一個 [main.py](#) 做這個作業題一, 如下圖所示, 目前的這個 [main.py](#) 主要是要做"讀 raw 檔 + 顯示原圖":



(2) 程式碼撰寫 & 補充:

code(包含註解)	為什麼要這樣用(寫的邏輯架構)
<pre># step 1:正確讀取 RAW 彩色影像(Sequential 格式) # 檔案格式: 512x512, RGB, Sequential(Planar) # 這支程式會讀 baboon.raw 和 lena.raw # 然後把它們轉成 R、G、B 三個平面, 再顯示出來 import numpy as np import matplotlib.pyplot as plt # 基本設定影像的寬跟高(512x512) width = 512 height = 512 # 一張彩色圖要的總 byte 數 = 寬 * 高 * 3(R,G,B 三個平面) total_bytes = width * height * 3 # 然後讀一張 RAW 圖片的流程是: # 1. 用 np.fromfile 把檔案整個讀進來(uint8, 0~255) # 2. 檢查檔案大小對不對 # 3. 切成 R、G、B 三段 # 4. 每段 reshape 成 (height, width) # 5. 組回一張 RGB 圖片顯示 # 做成一個小函式, 等等 baboon 跟 lena 都會用到 def read_raw_image(filename): print(f"開始讀取檔案: {filename}") # 1. 讀檔(binary), dtype=uint8 代表每個 byte 是 0~255 data = np.fromfile(filename, dtype=np.uint8) # 2. 檢查大小是否正確 if data.size != total_bytes: print(f"檔案大小不對! 讀到 {data.size} bytes, 但正常應該是 {total_bytes} bytes") # 這邊先直接 return, 避免後面 reshape 爆掉 return None, None, None, None # 3. Sequential 排列: 所有 R 在前面, 接著所有 G, 再來是 B # 前面 width*height 個是 R # 中間 width*height 個是 G # 後面 width*height 個是 B R_flat = data[0 : width * height] G_flat = data[width * height : 2 * width * height] B_flat = data[2 * width * height : 3 * width * height] # 4. 變成 2D 陣列(影像平面), 每個都是 512x512 R = R_flat.reshape((height, width))</pre>	- 這邊的 import numpy as np, 是因為numpy 是 Python 處理影像矩陣資料的核心工具, RAW 檔是一堆 byte(0~255 數字)組成, 需要拆成 R、G、B 三個 512*512 的矩陣。 - import matplotlib.pyplot as plt, 是要用來顯示圖片。 - 在第一部分:要先將基本設定寫定大小(Width、height 都寫成給好的 512), 然後會寫 total_bytes = width * height * 3的原因是因為一張圖有 512 * 512 個 pixel, 每個 pixel 有 3 channel (R、G、B), 每個 channel 1-byte, 所以總共 512*512*3-byte, 然後會用這個大小作確認檔案大小是否正確, 避免讀檔錯誤。 - 第二部分:有用 np.fromfile(filename, dtype=np.uint8), 這是因為 fromfile 可以直接把 binary 檔案讀成一個一維的陣列;然後 dtype=np.uint8 剛好是對應到 raw 裡每一個 byte, 是 0~255 範圍的灰階或顏色值;data.size != total_bytes 則是如果檔案大小不是預期的 512*512*3, 就代表不是我們想像中的圖(可能尺寸不同或檔案不完整), 這裡會印出警告, 然後 return None, 避免後面在 reshape時出錯。 - 第三部分:依作業指定的 RAW 採 Sequential 排列, 前 width*height 個 byte 會是整張圖的 R 平面, 接下來 width*height 個是整張圖的 G 平面, 最後 width*height 個是整張圖的 B 平面, 我是以直接切片的方法將一維陣列分成三段(而不是用 RGBRGBRGBRGB 的方式讀)。 - 第四部分:把一維陣列的 reshape 成 2D 影像, 其中 reshape((height, width)) 就是把長度 512*512 的一維陣列變成 512*512 的矩陣, 這樣每個 (y, x) 位置就對應到一個 pixel 的顏色值, 執行完這一步, 就能拿到R(R 通道的 2D 影像), G(G 通道的 2D 影像), B(B 通道的 2D 影像), 後面在做 RGB -> HSI, Histogram Equalization, Gamma 變換, 就可以直接針對這三個矩陣去進行運算。 - 第五部分:我會用np.dstack((R, G, B)) 把三個 2D 平面疊成一個 (height, width, 3) 的陣列, 這個格式剛好可以

```

G = G_flat.reshape((height, width))
B = B_flat.reshape((height, width))

# 5. 把三個平面疊在一起, 變成一張 RGB 彩色圖 (高,寬,3)
img_rgb = np.dstack((R, G, B))

print(f"讀取成功, R, G, B 的 shape 都是: {R.shape}")
return R, G, B, img_rgb
# 讀取 baboon.raw
R_baboon, G_baboon, B_baboon, img_baboon =
read_raw_image("baboon.raw")

if img_baboon is not None:
    plt.figure()
    plt.imshow(img_baboon)
    plt.title("Baboon (from baboon.raw)")
    plt.axis("off") # 不顯示座標
# 讀取 lena.raw
R_lena, G_lena, B_lena, img_lena = read_raw_image("lena.raw")

if img_lena is not None:
    plt.figure()
    plt.imshow(img_lena)
    plt.title("Lena (from lena.raw)")
    plt.axis("off")
# 顯示兩張圖 (如果上面都有讀成功)
plt.show()
# 到這裡為止:
# 已經拿到 R_baboon, G_baboon, B_baboon, R_lena, G_lena, B_lena
# 之後做 RGB -> HSI, HE, Gamma 都會用到它們

```

直接拿給matplotlib.pyplot.imshow() 顯示, 然後這一步的目的主要是檢查讀檔與切割是否正確, 如果顏色正常表示 Sequential 的切法沒問題, 或是如果顏色很怪或圖像扭曲也很可能是寬高或排列方式弄錯了。

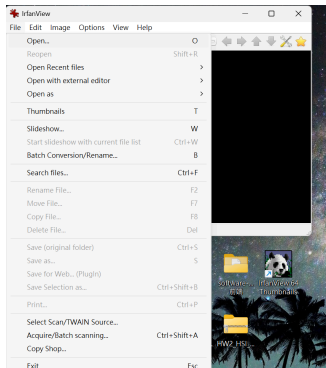
- 第六部分: 我用同一個 read_raw_image() 函式, 分別讀 baboon.raw、lena.raw, 如果回傳的 img_baboon / img_lena 不是 None, 就會用 plt.imshow() 顯示出來 (plt.axis("off") 會把座標軸關掉, 讓報告裡的圖比較乾淨), 這一步的目標主要是確認兩張測試圖都能正常讀取, 之後在結果測試更可以直接用這兩張原圖當作對照組, 方便比對: 原始 RGB 圖、只加強 I 的圖、S 做 Gamma 的圖、H 做特殊變化的圖。
- 這一部分做完後, 我已經擁有: R_baboon, G_baboon, B_baboon, R_lena, G_lena, B_lena, 這些都是 512×512 的 numpy 陣列, 每個元素都是 0~255 的 uint8 整數。

- 參考資料來源:

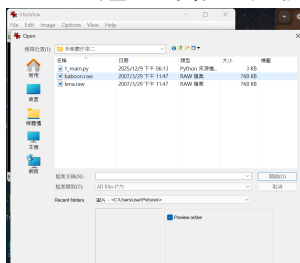
- NumPy 官方文件: fromfile, 如何讀取 binary RAW 資料
<https://numpy.org/doc/stable/reference/generated/numpy.fromfile.html>
- NumPy 官方文件: reshape — 將一維 RAW 轉成 512×512
<https://numpy.org/doc/stable/reference/generated/numpy.reshape.html>
- 協助與偵錯: 程式邏輯以及撰寫皆由本人先行完成, 再使用 ChatGPT 協助檢查錯誤。
- (額外) 用老師提及的 IrfanView 64 (上面紅色怪獸 icon) 作為 RAW 彩色影像檢查工具:

步驟(及解說):

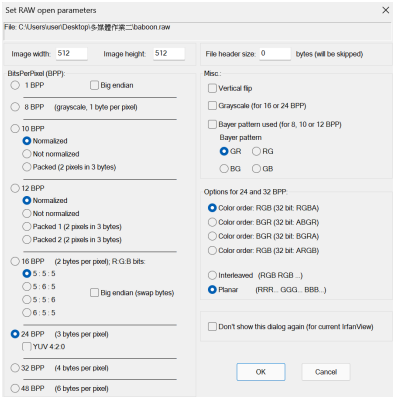
1. 打開下載好的 IrfanView 64 後按 FILE



2. 選老師給的兩個.raw檔案



3. 按照老師的要求填寫資料:

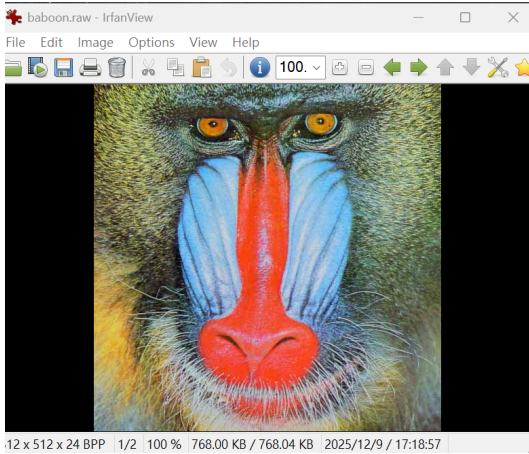


Width	512
Height	512
Header	0
Bits Per Pixel	24 BPP
Color order	RGB
Format	Planar (RRR...GGG...BBB...)

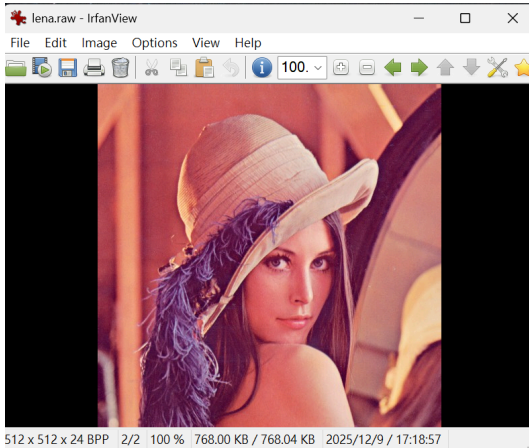
- 結果1(比對, 可以做double check, 我在切 R/G/B 的方式沒有錯):

IrfanView 64

- baboon.raw

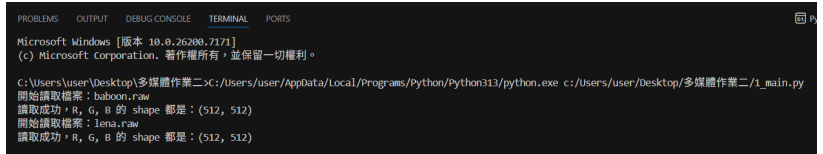


- lena.raw

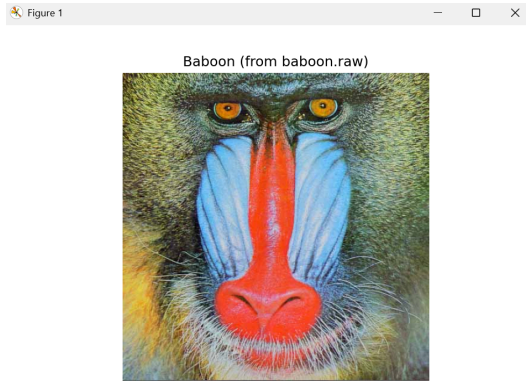


我撰寫的程式結果
plt.imshow(img_baboon)
plt.show()

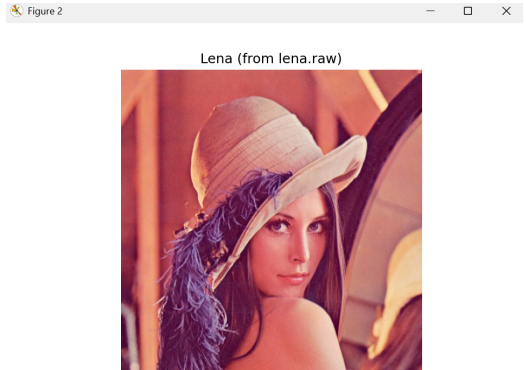
- bs code's terminal



- baboon.raw



- lena.raw



- 作業題目二_Step 2、6:接著做RGB => HSI 轉換、最後的HSI => RGB 轉換
- 實作方法:因為老師在課堂以及作業 ppt 都有放轉換公式(如下圖所截), 所以在這一部分我只是將轉換數學公式轉換為 code 撰寫, 就沒有要特別詳細的說明

H.S.I 彩色轉換



$$\theta = \cos^{-1} \left\{ \frac{\frac{1}{2}[(R-G) + (R-B)]}{\left[(R-G)^2 + (R-B)(G-B) \right]^{1/2}} \right\}$$

$$H = \begin{cases} \theta & \text{if } B \leq G \\ 360 - \theta & \text{if } B > G \end{cases}$$

$$S = 1 - \frac{3}{(R+G+B)} [\min(R, G, B)]$$

$$I = \frac{1}{3}(R+G+B)$$

If $0^\circ \leq H < 120^\circ$,

$$R = I \left[1 + \frac{S \cos H}{\cos(60^\circ - H)} \right]$$

$$G = 3I - (R + B)$$

$$B = I(1 - S)$$

If $120^\circ \leq H < 240^\circ$, $H = H - 120$

$$R = I(1 - S)$$

$$G = I \left[1 + \frac{S \cos H}{\cos(60^\circ - H)} \right]$$

$$B = 3I - (R + G)$$

If $240^\circ \leq H \leq 360^\circ$, $H = H - 240$

$$R = 3I - (G + B)$$

$$G = I(1 - S)$$

$$B = I \left[1 + \frac{S \cos H}{\cos(60^\circ - H)} \right]$$

建議以上RGB值在實作時應取
Value = min(255, floor(...))
 以避免overflow之後值變很小

3

- (1) 新增 rgb_to_hsi.py、hsi_to_rgb.py 做撰寫(另外我一開始的 [main.py](#) 也要做對應新增 code)
- (2) 在這個 rgb_to_hsi.py 做完轉換後, 最後會回傳三種 channel: **H** (0~360 度)、**S** (0~1)、**I** (0~255 (亮度)), 以及 hsi_to_rgb.py 做完轉換後會將後續有做修改的 HSI 轉回 RGB (會用用 matplotlib 顯示做完修改的結果影像), 後面 step 3 ~ step 5 都會用到。
- (3) rgb_to_hsi.py 程式碼撰寫(含註解) & 補充:

main.py (新增轉換 rgb_to_hsi.py 的呼叫)

```
# step 1:正確讀取 RAW 彩色影像 (Sequential 格式)
# 檔案格式: 512x512, RGB, Sequential (Planar)
# 這支程式會讀 baboon.raw 和 lena.raw
# 然後把它們轉成 R、G、B 三個平面, 再顯示出來
import numpy as np
import matplotlib.pyplot as plt

# step 2: RGB 轉 HSI 呼叫
from rgb_to_hsi import rgb_to_hsi

# 影像的寬跟高 (512x512)
width = 512
height = 512
# 一張彩色圖要的總 byte 數 = 寬 * 高 * 3 (R,G,B 三個平面)
total_bytes = width * height * 3
# 然後讀一張 RAW 圖片的流程是:
# 1. 用 np.fromfile 把檔案整個讀進來 (uint8, 0~255)
# 2. 檢查檔案大小對不對
# 3. 切成 R、G、B 三段
# 4. 每段 reshape 成 (height, width)
# 5. 組回一張 RGB 圖片顯示

# 做成一個小函式, 等等 baboon 跟 lena 都會用到
def read_raw_image(filename):
    print(f"開始讀取檔案: {filename}")

    # 1. 讀檔 (binary), dtype=uint8 代表每個 byte 是 0~255
    data = np.fromfile(filename, dtype=np.uint8)

    # 2. 檢查大小是否正確
    if data.size != total_bytes:
        print(f"檔案大小不對! 讀到 {data.size} bytes, 但正常應該是 {total_bytes} bytes")
        # 這邊先直接 return, 避免後面 reshape 爆掉
        return None, None, None, None

    # 3. Sequential 排列: 所有 R 在前面, 接著所有 G, 再來是 B
    # 前面 width*height 個是 R
    # 中間 width*height 個是 G
    # 後面 width*height 個是 B
    R_flat = data[0 : width * height]
    G_flat = data[width * height : 2 * width * height]
```

rgb_to_hsi.py

```
# step 2: RGB 轉 HSI
# 這部份程式是依照老師 PPT 上的公式, 把 R/G/B 轉成 H/S/I 三個通道
# 因為之後 Step 3、Step 4、Step 5 都會用到 H/S/I
import numpy as np

def rgb_to_hsi(R, G, B):
    # 1. 先把 R, G, B 轉成浮點數, 避免後面公式計算被整數截斷
    R = R.astype(np.float32)
    G = G.astype(np.float32)
    B = B.astype(np.float32)

    # 2. 計算 I (Intensity)
    # I = (R + G + B) / 3
    I = (R + G + B) / 3.0

    # 3. 計算 S (Saturation)
    # 如果 R、G、B 三個最小值加起來為 0, S 會變成 0, 避免除以 0 錯誤
    min_rgb = np.minimum(np.minimum(R, G), B)
    sum_rgb = R + G + B
    # 4. S = 1 - 3 * min(R,G,B) / (R+G+B)
    # 如果 sum = 0, 就直接設成 0
    S = np.where(sum_rgb == 0, 0, 1 - (3 * min_rgb / sum_rgb))

    # 5. 計算 H (Hue)
    # 老師 PPT 的公式:
    # θ = arccos{ [(R-G)+(R-B)] / [2*sqrt((R-G)^2+(R-B)(G-B))] }
    # 若 B <= G, H = θ
    # 若 B > G, H = 360° - θ

    # 分子 numerator
    numerator = ( (R - G) + (R - B) ) / 2.0
    # 分母 denominator
    denominator = np.sqrt( (R - G)**2 + (R - B)*(G - B) )
    # 避免除以零
    denominator = np.where(denominator == 0, 1e-6, denominator)
    # acos 的輸入要介於 [-1, 1]
    cos_theta = numerator / denominator
    cos_theta = np.clip(cos_theta, -1, 1)
    # 計算 θ (單位是弧度 rad)
    theta = np.arccos(cos_theta)
    # H 根據 B <= G 或 B > G 分成兩種情況
    H = np.where(B <= G, theta, 2*np.pi - theta)
    # 把 H 從弧度轉成角度
    H = H * 180 / np.pi
```

```

B_flat = data[2 * width * height : 3 * width * height]

# 4. 變成 2D 陣列 (影像平面). 每個都是 512x512
R = R_flat.reshape((height, width))
G = G_flat.reshape((height, width))
B = B_flat.reshape((height, width))

# 5. 把三個平面疊在一起. 變成一張 RGB 彩色圖 (高,寬,3)
img_rgb = np.dstack((R, G, B))

print(f"讀取成功, R, G, B 的 shape 都是: {R.shape}")
return R, G, B, img_rgb

# 讀取 baboon.raw
R_baboon, G_baboon, B_baboon, img_baboon =
read_raw_image("baboon.raw")

if img_baboon is not None:
    plt.figure()
    plt.imshow(img_baboon)
    plt.title("Baboon (from baboon.raw)")
    plt.axis("off") # 不顯示座標
# 讀取 lena.raw
R_lena, G_lena, B_lena, img_lena = read_raw_image("lena.raw")

if img_lena is not None:
    plt.figure()
    plt.imshow(img_lena)
    plt.title("Lena (from lena.raw)")
    plt.axis("off")
# 顯示兩張圖 (如果上面都有讀成功)
plt.show()
# 到這裡為止:
# 已經拿到 R_baboon, G_baboon, B_baboon, R_lena, G_lena, B_lena
# 之後做 RGB -> HSI, HE, Gamma 都會用到它們

# step 2: RGB 轉 HSI 呼叫
# 把 baboon RAW 轉成 HSI
H_b, S_b, I_b = rgb_to_hsi(R_baboon, G_baboon, B_baboon)
print("baboon HSI 轉換完成")
# 把 lena RAW 轉成 HSI
H_l, S_l, I_l = rgb_to_hsi(R_lena, G_lena, B_lena)
print("lena HSI 轉換完成")

```

```
return H, S, I
```

補充:有用藍底標的是新增的。

補充:

1. 在程式中:

```

R = R.astype(np.float32)
G = G.astype(np.float32)
B = B.astype(np.float32)

```

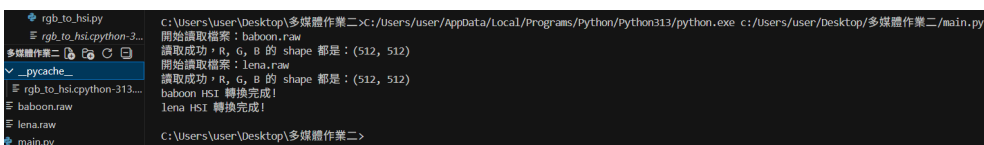
RAW 的 RGB 原本是 uint8 (0~255), 如果不轉 float , 做除法會被截斷(整數除法), 且算飽和度 S 會錯, 還有算 acos (色調) 時會超出範圍, 整張 HSI 都會不準, 所以先轉 float32 才能進行數學計算。

2. 在程式中:

```
cos_theta = np.clip(cos_theta, -1, 1)
```

計算 H (色調)的這一行要 clip 是因為cos_theta 可能出現 1.00000002 or -1.00000002 , 而 arccos() 的輸入只能介於 [-1, 1], 不然程式會噴爆(可怕)。

● 結果 2:



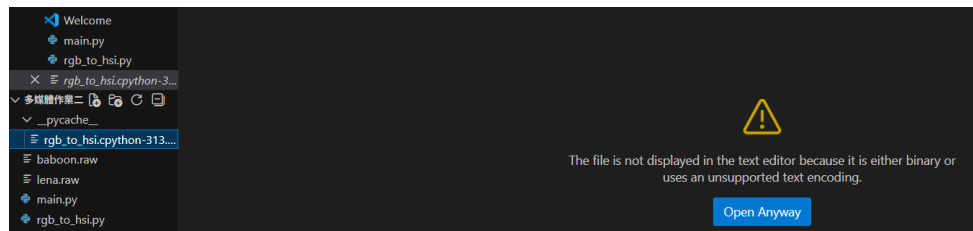
```

C:\Users\user\Desktop\多媒體作業二>C:\Users\user\AppData\Local\programs\Python\Python313\python.exe c:/Users/user/Desktop/多媒體作業二/main.py
開始讀取檔案: baboon.raw
讀取成功, R, G, B 的 shape 都是: (512, 512)
開始讀取檔案: lena.raw
讀取成功, R, G, B 的 shape 都是: (512, 512)
baboon HSI 轉換完成!
lena HSI 轉換完成!
C:\Users\user\Desktop\多媒體作業二>

```

再執行一次 [main.py](#), 除了會再產一次圖之外 (結果一的圖), 也會將其的 RGB 轉成 HSI, 存起來。

terminal 會多印出:
baboon HSI 轉換完成
lena HSI 轉換完成



因為我在 main.py 裡寫了：
from rgb_to_hsi import rgb_to_hsi
程式會做兩件事：
讀取你寫的 rgb_to_hsi.py
產生一個編譯版本
__pycache__/rgb_to_hsi.cpython-313.pyc
這樣下次你再跑 main.py 的時候，Python 不需要重新解析你的原始碼，
可以直接用快取，加快執行速度。

(4) hsi_to_rgb.py 程式碼撰寫(含註解) & 補充:

hsi_to_rgb.py

```
# step 6:將 H, S, I 轉回 R, G, B
# 公式來自課堂 HSI 轉 RGB 定義(老師 PPT)
import numpy as np

def hsi_to_rgb(H, S, I):
    # 把角度轉回弧度(公式要使用弧度)
    H = H * np.pi / 180.0
    R = np.zeros_like(H)
    G = np.zeros_like(H)
    B = np.zeros_like(H)
    # 三段 H 區間(因為 RGB 分段定義)
    # 0 ~ 120度
    idx1 = (H >= 0) & (H < 2*np.pi/3)
    # 120 ~ 240度
    idx2 = (H >= 2*np.pi/3) & (H < 4*np.pi/3)
    # 240° ~ 360°
    idx3 = (H >= 4*np.pi/3) & (H < 2*np.pi)
    # 第一段: 0~120度
    H1 = H[idx1]
    S1 = S[idx1]
    I1 = I[idx1]
    B[idx1] = I1 * (1 - S1)
    R[idx1] = I1 * (1 + (S1 * np.cos(H1) / np.cos(np.pi/3 - H1)))
    G[idx1] = 3*I1 - (R[idx1] + B[idx1])
    # 第二段: 120~240度
    H2 = H[idx2] - 2*np.pi/3
    S2 = S[idx2]
    I2 = I[idx2]
    R[idx2] = I2 * (1 - S2)
    G[idx2] = I2 * (1 + (S2 * np.cos(H2) / np.cos(np.pi/3 - H2)))
    B[idx2] = 3*I2 - (R[idx2] + G[idx2])
    # 第三段: 240~360度
    H3 = H[idx3] - 4*np.pi/3
    S3 = S[idx3]
    I3 = I[idx3]
    G[idx3] = I3 * (1 - S3)
    B[idx3] = I3 * (1 + (S3 * np.cos(H3) / np.cos(np.pi/3 - H3)))
    R[idx3] = 3*I3 - (G[idx3] + B[idx3])
    # 裁切到合法範圍
    R = np.clip(R, 0, 255).astype(np.uint8)
    G = np.clip(G, 0, 255).astype(np.uint8)
    B = np.clip(B, 0, 255).astype(np.uint8)
    return R, G, B
```

- 參考資料來源:

- np.clip():
<https://blog.csdn.net/qg1483661204/article/details/78150203>
<https://numpy.org/doc/stable/reference/generated/numpy.clip.html>
- 協助與偵錯:程式邏輯以及撰寫皆由本人先行完成，再使用ChatGPT協助檢查錯誤。

- 作業題目三_Step 3:以 Histogram Equalization 進行 I channel 的加強:

- 實作方法:為了加強影像亮度的對比, 我對從 HSI 色彩空間取得的 I(Intensity)通道做直方圖均衡化, 這樣能讓原本分布過度集中於暗部或亮部的像素, 重新分布到較均勻的範圍, 讓影像的細節變得更加清楚。
- (1) 新增 hist_eq_l.py 做 code 撰寫。
 - (2) 會計算 I 通道的直方圖(0~255)來統計每個亮度值出現的次數。
 - (3) 再計算累積分布 CDF 將 histogram 的累積結果正規化到 0~1。
 - (4) 建立新的亮度對應表會使用 $\text{new_value} = \text{CDF}[\text{value}] \times 255$, 會把舊亮度映射到新的亮度。
 - (5) 再依照 mapping 替換原本的 I 通道, 得到亮度對比提升後的新 I' 通道。
 - (6) 做完 HE 之後, 就會得到 I_equalized, 它之後會與 H、S 一起用於 HSI -> RGB 的重建部分。
 - (7) 程式碼撰寫 & 補充:

hist_eq_l.py	main.py (新增 hist_eq_l.py、轉換 rgb_to_hsi.py 的呼叫)
<pre> # Step 3:對 I 通道進行 Histogram Equalization(直方圖均衡化) # 這支程式的目的: # 把從 HSI 得到的 I (亮度) 通道做對比度提升 # 讓暗的地方變亮、亮的地方變暗, 使整體亮度分布更平均 # 直方圖均衡化主要流程: # 1. 計算 I channel 每個亮度值(0~255)出現的次數 → histogram # 2. 計算累積分布函數 CDF (每個亮度以下的累積機率) # 3. 把舊 I 值映射到新的值: new_I = CDF[I] * 255 import numpy as np def hist_equalize_I(I): # I 是 float 值 (由 RGB-HSI 得來), 但 HE 需要用 0~255 的整數亮度 # 所以先做 clipping + 轉成 uint8 I_uint8 = np.clip(I, 0, 255).astype(np.uint8) # 1. 計算 histogram (0~255 共 256 個 bin) hist, bins = np.histogram(I_uint8, bins=256, range=(0, 255)) # 2. 計算累積分布函數 CDF cdf = hist.cumsum() # 累積到目前亮度為止的像素數量 cdf_normalized = cdf / cdf[-1] # 除以最後一個累積值, 正規化到 0~1 # 3. 建立新的 mapping: 把亮度值 0~255 映射到新亮度 # new_value = CDF[value] * 255 new_values = (cdf_normalized * 255).astype(np.uint8) # 4. 套用 mapping 取代舊 I 值 I_equalized = new_values[I_uint8] # 轉回 float32 (後面還會和 S、H 一起轉回 RGB) return I_equalized.astype(np.float32) </pre>	<pre> # step 1:正確讀取 RAW 彩色影像(Sequential 格式) # 檔案格式: 512x512, RGB, Sequential(Planar) # 這支程式會讀 baboon.raw 和 lena.raw # 然後把它們轉成 R、G、B 三個平面, 再顯示出來 import numpy as np import matplotlib.pyplot as plt # step 2: RGB 轉 HSI 呼叫 from rgb_to_hsi import rgb_to_hsi # step 3: 呼叫以 Histogram Equalization 進行 i channel 的加強 from hist_eq_l import hist_equalize_I # step 6: HSI 轉 RGB (顯示 HE 成果) from hsi_to_rgb import hsi_to_rgb # 影像的寬跟高(512x512) width = 512 height = 512 # 一張彩色圖案的總 byte 數 = 寬 * 高 * 3(R,G,B 三個平面) total_bytes = width * height * 3 # 然後讀一張 RAW 圖片的流程是: # 1. 用 np.fromfile 把檔案整個讀進來(uint8, 0~255) # 2. 檢查檔案大小對不對 # 3. 切成 R、G、B 三段 # 4. 每段 reshape 成 (height, width) # 5. 組回一張 RGB 圖片顯示 # 做成一個小函式, 等等 baboon 跟 lena 都會用到 def read_raw_image(filename): print(f"開始讀取檔案: {filename}") # 1. 讀檔(binary), dtype=uint8 代表每個 byte 是 0~255 data = np.fromfile(filename, dtype=np.uint8) # 2. 檢查大小是否正確 if data.size != total_bytes: print(f"檔案大小不對! 讀到 {data.size} bytes, 但正常應該是 {total_bytes} bytes") # 這邊先直接 return, 避免後面 reshape 爆掉 return None, None, None, None # 3. Sequential 排列: 所有 R 在前面, 接著所有 G, 再來是 B # 前面 width*height 個是 R # 中間 width*height 個是 G # 後面 width*height 個是 B R_flat = data[0 : width * height] G_flat = data[width * height : 2 * width * height] B_flat = data[2 * width * height : 3 * width * height] # 4. 變成 2D 陣列(影像平面), 每個都是 512x512 R = R_flat.reshape((height, width)) G = G_flat.reshape((height, width)) B = B_flat.reshape((height, width)) # 5. 把三個平面疊在一起, 變成一張 RGB 彩色圖(高,寬,3) img_rgb = np.dstack((R, G, B)) </pre>

```

print(f"讀取成功, R, G, B 的 shape 都是:{R.shape}")
return R, G, B, img_rgb
# 讀取 baboon.raw
R_baboon, G_baboon, B_baboon, img_baboon =
read_raw_image("baboon.raw")

if img_baboon is not None:
    plt.figure()
    plt.imshow(img_baboon)
    plt.title("Baboon (from baboon.raw)")
    plt.axis("off") # 不顯示座標
# 讀取 lena.raw
R_lena, G_lena, B_lena, img_lena = read_raw_image("lena.raw")

if img_lena is not None:
    plt.figure()
    plt.imshow(img_lena)
    plt.title("Lena (from lena.raw)")
    plt.axis("off")
# 顯示兩張圖(如果上面都有讀成功)
plt.show()
# 到這裡為止:
# 已經拿到 R_baboon, G_baboon, B_baboon, R_lena, G_lena, B_lena
# 之後做 RGB -> HSI、HE、Gamma 都會用到它們

# step 2: RGB 轉 HSI 呼叫
# 把 baboon RAW 轉成 HSI
H_b, S_b, I_b = rgb_to_hsi(R_baboon, G_baboon, B_baboon)
print("baboon HSI 轉換完成")
# 把 lena RAW 轉成 HSI
H_l, S_l, I_l = rgb_to_hsi(R_lena, G_lena, B_lena)
print("lena HSI 轉換完成")

# step 3: 呼叫以 Histogram Equalization 進行 I channel 的加強
# 對 baboon 做 HE
# step 3-1: I 通道 HE
I_b_he = hist_equalize(I_b)
I_l_he = hist_equalize(I_l)

# step 3-2: 手動調整亮度(可調)
brightness_factor = 0.4 # >1 加亮; <1 變暗; 1 不變

# baboon
I_b_new = np.clip(I_b_he * brightness_factor, 0, 255)
print("baboon I 通道直方圖均衡化完成!")

# lena
I_l_new = np.clip(I_l_he * brightness_factor, 0, 255)
print("lena I 通道直方圖均衡化完成!")

# step 6-1: HSI 轉 RGB (顯示 HE 成果)
# 把新的 I (I_b_new) 與原本 H_b, S_b 組在一起轉回 RGB
R_b2, G_b2, B_b2 = hsi_to_rgb(H_b, S_b, I_b_new)
img_b2 = np.dstack((R_b2, G_b2, B_b2))

plt.figure()
plt.imshow(img_b2)
plt.title("Baboon - I channel Histogram Equalization")
plt.axis("off")

# lena 也做一次
R_l2, G_l2, B_l2 = hsi_to_rgb(H_l, S_l, I_l_new)
img_l2 = np.dstack((R_l2, G_l2, B_l2))

plt.figure()
plt.imshow(img_l2)
plt.title("Lena - I channel Histogram Equalization")
plt.axis("off")
plt.show()

```

- 為什麼要這樣用(寫的邏輯架構):
1. 目的:重新分配亮度值, 使影像亮度分布更平均, 提升暗部/亮部細節。
 2. 重點:HSI 轉換中的 I 通道代表亮度, 所以加

補充:有用藍底標的是新增的。
step 3-2: 手動調整亮度(可調)
brightness_factor = 1.2 # >1 加亮; <1 變暗; 1 不變

強亮度最佳方法就是針對 I 做 HE。

3. HE 的核心流程:統計亮度分布、計算累積分布(CDF)、把舊亮度映射到新的亮度值、得到對比度提升後的 I' 通道, 這樣會讓整張圖的亮暗分布變得更平均, 使得原本看起來灰濛濛的部分變更清晰。
4. 第一部分:要 clip + 轉 uint8, 是因為直方圖均衡化是依亮度級別(0~255)去進行統計, 因此需將 I 通道轉成 uint8 型態, 並以 clip 限於合理範圍內。
5. 第二部分:計算 histogram, 是均衡化計算的基礎, 用來了解影像亮度的集中程度。
6. 第三部分:計算 CDF, 是因為它能让原本集中的亮度分布變成平均展開, CDF 是從 0 級到目前級的累積機率, 最後一個值 $\text{cdf}[-1]$ 是總像素數(用來正規化), 要除以 $\text{cdf}[-1]$ 是因為 CDF 最大值 = 影像總像素數, 除以它之後所有 CDF 值會在 0~1 之間才能拿去乘 255 做亮度映射。
7. 第四部份: $\text{new_value} = \text{CDF} * 255$, 是為了讓亮度會展開、均勻化, 以增加細節。

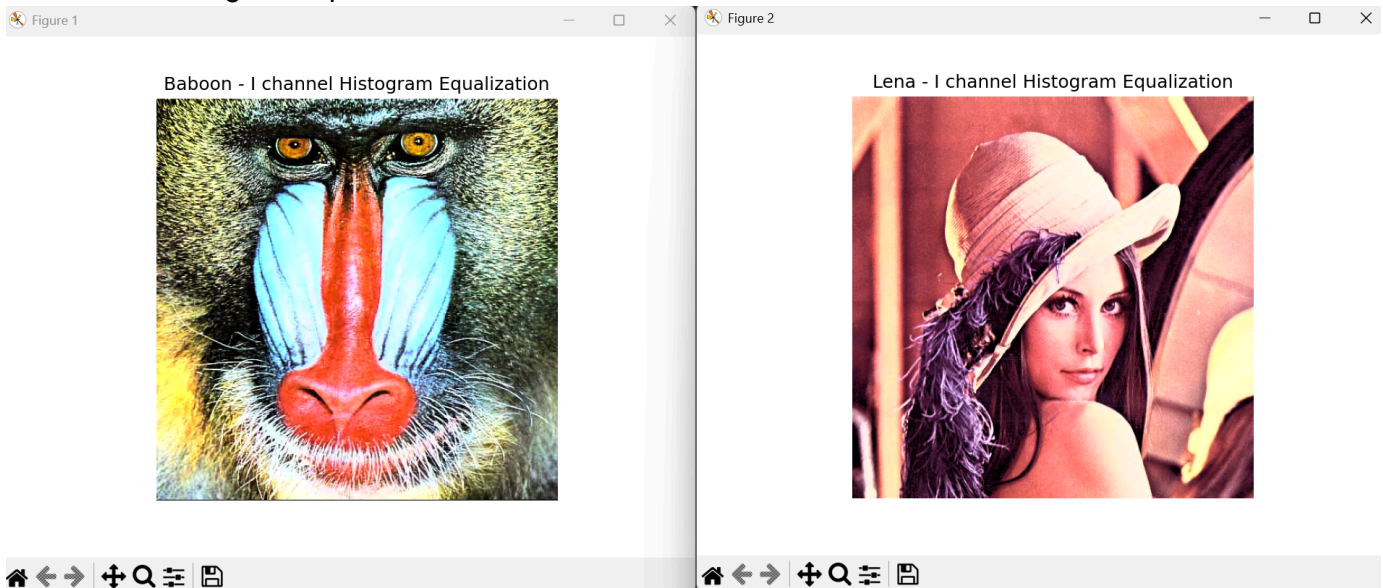
- 結果 3:

- terminal:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Microsoft Windows [版本 10.0.26200.7171]
(c) Microsoft Corporation. 著作權所有, 並保留一切權利。

C:\Users\user\Desktop\多媒體作業二>C:\Users\user\AppData\Local\Programs\Python\Python313\python.exe c:/Users/user/Desktop/多媒體作業二/main.py
開始讀取檔案: baboon.raw
讀取成功, R, G, B 的 shape 都是: (512, 512)
開始讀取檔案: lena.raw
讀取成功, R, G, B 的 shape 都是: (512, 512)
baboon HSI 轉換完成
lena HSI 轉換完成
baboon I 通道直方圖均衡化完成!
lena I 通道直方圖均衡化完成!
```

- 經過Histogram Equalization 進行 I channel = 1.2 的加強後的影像:



- 參考資料來源:

1. Histogram Equalization 原理(OpenCV 官方教學):
https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html
 2. NumPy 官方文件 — histogram() 函式:
<https://numpy.org/doc/stable/reference/generated/numpy.histogram.html>
 3. NumPy 官方文件 —
cumsum():<https://numpy.org/doc/stable/reference/generated/numpy.cumsum.html>
 4. np.clip():
<https://blog.csdn.net/qg1483661204/article/details/78150203>
<https://numpy.org/doc/stable/reference/generated/numpy.clip.html>
 5. 協助與偵錯:程式邏輯以及撰寫皆由本人先行完成, 再使用ChatGPT協助檢查錯誤。
- 作業題目四_Step 4:以 **Gamma Transformation** 進行 **S channel** 的加強:
 - 實作方法:為了加強影像的色彩飽和度, 我對從 HSI 色彩空間取得的 S 通道進行 Gamma Transformation, 飽和度代表顏色的鮮豔程度, 所以會透過 Gamma 的方式調整 S 值, 可以讓整張影像的色彩更加突出或更加淡漠, 視所選擇的 gamma 值而定, 若選擇 $\text{gamma} < 1$, 色彩會變得更飽和、更加鮮豔;若 $\text{gamma} > 1$, 則可降低飽和度, 使畫面較為淡漠柔和。
 - (1) 新增gamma_S.py 來撰寫
 - (2) Gamma Transformation 的公式為: $S' = S^{\text{gamma}}$, gamma 用來控制色彩強化程度。
 - (3) 若選擇 $\text{gamma} < 1$, 色彩會變得更飽和、更加鮮豔;若 $\text{gamma} > 1$, 則可降低飽和度, 使畫面較為淡漠柔和。
 - (4) 得到新的 S' (S_new)後, 會用HSI 轉 RGB 重建。
 - (5) 程式碼撰寫 & 補充:

gamma_S.py	main.py (新增 gamma_S.py、轉換 rgb_to_hsi.py 的呼叫)
<pre> # Step 4: 對 S 通道做 Gamma Transformation (飽和度加強) # Gamma 的公式: # S_new = S^(gamma) # 如果 gamma < 1, 顏色會變得更飽和、更鮮豔 # 如果 gamma > 1, 顏色會變淡、變灰 # S 在 HSI 是 0~1, 所以用 S^(gamma) 就可以了 import numpy as np def gamma_transform_S(S, gamma): # 保證 S 在 0~1 之間 (避免超出範圍) S = np.clip(S, 0, 1) # 套用 Gamma 公式 S_new = S ** gamma # 轉回 float32 (後面 HSI 轉 RGB 要用) return S_new.astype(np.float32) </pre>	<pre> # step 1: 正確讀取 RAW 彩色影像(Sequential 格式) # 檔案格式: 512x512, RGB, Sequential(Planar) # 這支程式會讀 baboon.raw 和 lena.raw # 然後把它們轉成 R、G、B 三個平面, 再顯示出來 import numpy as np import matplotlib.pyplot as plt # step 2: RGB 轉 HSI 呼叫 from rgb_to_hsi import rgb_to_hsi # step 3: 呼叫以 Histogram Equalization 進行 I channel 的加強 from hist_eq_I import hist_equalize_I # step 4: 呼叫 S 通道做 Gamma Transformation from gamma_S import gamma_transform_S # step 6: HSI 轉 RGB (顯示 HE、S Gamma 的結果)) from hsi_to_rgb import hsi_to_rgb # 影像的寬跟高(512x512) width = 512 height = 512 # 一張彩色圖要的總 byte 數 = 寬 * 高 * 3(R,G,B 三個平面) total_bytes = width * height * 3 # 然後讀一張 RAW 圖片的流程是: # 1. 用 np.fromfile 把檔案整個讀進來(uint8, 0~255) # 2. 檢查檔案大小對不對 # 3. 切成 R、G、B 三段 # 4. 每段 reshape 成 (height, width) # 5. 組回一張 RGB 圖片顯示 # 做成一個小函式, 等等 baboon 跟 lena 都會用到 def read_raw_image(filename): print(f"開始讀取檔案: {filename}") # 1. 讀檔(binary), dtype=uint8 代表每個 byte 是 0~255 data = np.fromfile(filename, dtype=np.uint8) # 2. 檢查大小是否正確 </pre>

```

if data.size != total_bytes:
    print(f"檔案大小不對! 讀到 {data.size} bytes, 但正常應該是 {total_bytes} bytes")
    # 這邊先直接 return, 避免後面 reshape 爆掉
    return None, None, None, None

# 3. Sequential 排列: 所有 R 在前面, 接著所有 G, 再來是 B
# 前面 width*height 個是 R
# 中間 width*height 個是 G
# 後面 width*height 個是 B
R_flat = data[0 : width * height]
G_flat = data[width * height : 2 * width * height]
B_flat = data[2 * width * height : 3 * width * height]

# 4. 變成 2D 陣列(影像平面), 每個都是 512x512
R = R_flat.reshape((height, width))
G = G_flat.reshape((height, width))
B = B_flat.reshape((height, width))

# 5. 把三個平面疊在一起, 變成一張 RGB 彩色圖 (高,寬,3)
img_rgb = np.dstack((R, G, B))

print(f"讀取成功, R, G, B 的 shape 都是: {R.shape}")
return R, G, B, img_rgb
# 讀取 baboon.raw
R_baboon, G_baboon, B_baboon, img_baboon =
read_raw_image("baboon.raw")

if img_baboon is not None:
    plt.figure()
    plt.imshow(img_baboon)
    plt.title("Baboon (from baboon.raw)")
    plt.axis("off") # 不顯示座標
# 讀取 lena.raw
R_lena, G_lena, B_lena, img_lena = read_raw_image("lena.raw")

if img_lena is not None:
    plt.figure()
    plt.imshow(img_lena)
    plt.title("Lena (from lena.raw)")
    plt.axis("off")

# 顯示兩張圖(如果上面都有讀成功)
plt.show()
# 到這裡為止:
# 已經拿到 R_baboon, G_baboon, B_baboon, R_lena, G_lena, B_lena
# 之後做 RGB -> HSI、HE、Gamma 都會用到它們

# step 2: RGB 轉 HSI 呼叫
# 把 baboon RAW 轉成 HSI
H_b, S_b, I_b = rgb_to_hsi(R_baboon, G_baboon, B_baboon)
print("baboon HSI 轉換完成")
# 把 lena RAW 轉成 HSI
H_l, S_l, I_l = rgb_to_hsi(R_lena, G_lena, B_lena)
print("lena HSI 轉換完成")

# step 3: 呼叫以 Histogram Equalization 進行 I channel 的加強
# 對 baboon 做 HE
# step 3-1: I 通道 HE
I_b_he = hist_equalize_I(I_b)
I_l_he = hist_equalize_I(I_l)

# step 3-2: 手動調整亮度(可調)
brightness_factor = 0.4 # >1 加亮: <1 變暗: 1 不變

# baboon
I_b_new = np.clip(I_b_he * brightness_factor, 0, 255)
print("baboon I 通道直方圖均衡化完成!")

# lena
I_l_new = np.clip(I_l_he * brightness_factor, 0, 255)
print("lena I 通道直方圖均衡化完成!")

# step 6-1: HSI 轉 RGB (顯示 HE 成果)
# 把新的 I (I_b_new) 與原本 H_b, S_b 組在一起轉回 RGB
R_b2, G_b2, B_b2 = hsi_to_rgb(H_b, S_b, I_b_new)

```

	<pre>img_b2 = np.dstack((R_b2, G_b2, B_b2)) plt.figure() plt.imshow(img_b2) plt.title("Baboon - I channel Histogram Equalization") plt.axis("off") # lena 也做一次 R_l2, G_l2, B_l2 = hsi_to_rgb(H_l, S_l, I_l_new) img_l2 = np.dstack((R_l2, G_l2, B_l2)) plt.figure() plt.imshow(img_l2) plt.title("Lena - I channel Histogram Equalization") plt.axis("off") plt.show() #step 4: 呼叫 S 通道做 Gamma Transformation gamma_value = 0.25 # 你可以自己調, <1 會更鮮豔 # baboon S_b_new = gamma_transform_S(S_b, gamma_value) print("baboon S 通道 Gamma 完成!") # lena S_l_new = gamma_transform_S(S_l, gamma_value) print("lena S 通道 Gamma 完成!") # step 6-2: HSI 轉 RGB (顯示 S Gamma 的結果) # baboon, 用 S_new + I (原始 I) R_b3, G_b3, B_b3 = hsi_to_rgb(H_b, S_b_new, I_b) img_b3 = np.dstack((R_b3, G_b3, B_b3)) plt.figure() plt.imshow(img_b3) plt.title("Baboon - S channel Gamma Transformation") plt.axis("off") # lena R_l3, G_l3, B_l3 = hsi_to_rgb(H_l, S_l_new, I_l) img_l3 = np.dstack((R_l3, G_l3, B_l3)) plt.figure() plt.imshow(img_l3) plt.title("Lena - S channel Gamma Transformation") plt.axis("off") plt.show()</pre>
為什麼要這樣用(寫的邏輯架構): 1. 重點: S (飽和度)範圍為 0~1, 用 Gamma 函數調整最合適, 使用公式: $S' = S^{\gamma}$: $\gamma < 1$, 增加飽和度, 顏色更鮮豔 $\gamma > 1$, 降低飽和度, 顏色變柔和 先 clip(S, 0, 1), 避免浮點誤差造成 S 超出範圍。 直接用 Gamma, 不需 histogram, 因為 S 不是亮度分布。 最後轉成 float32, 以便後續 HSI 轉 RGB 的計算。	補充:有用藍底標的是新增的。
結果 4:	terminal:

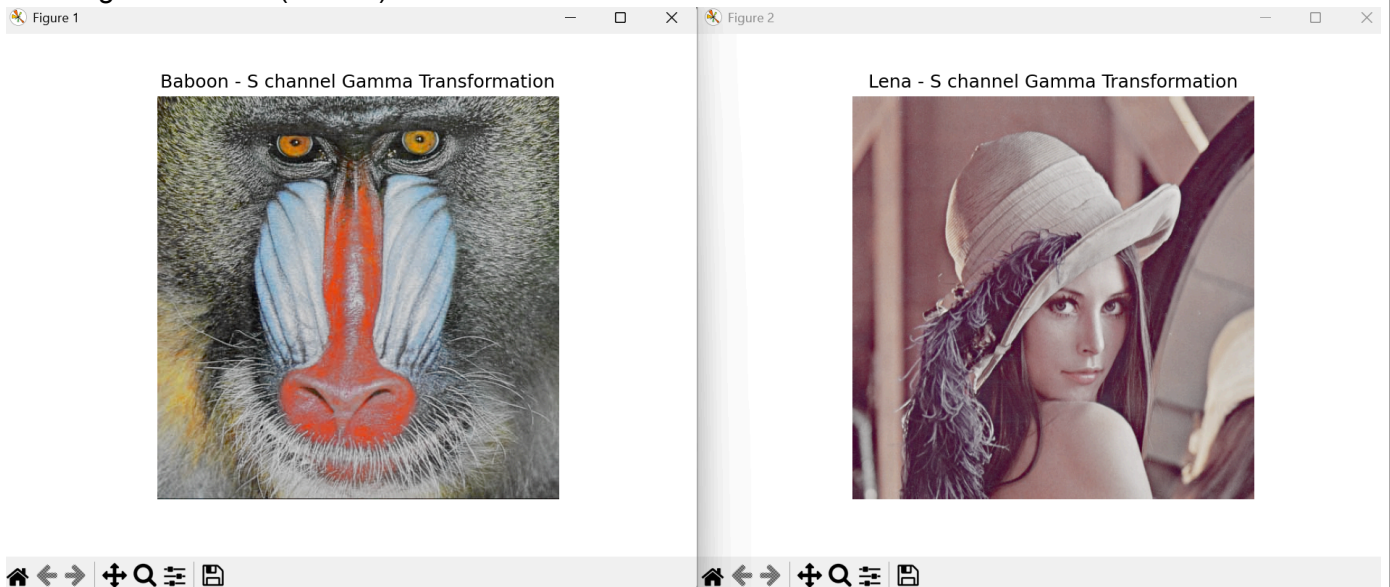
```

Microsoft Windows [版本 10.0.26200.7171]
(c) Microsoft Corporation. 著作權所有，並保留一切權利。

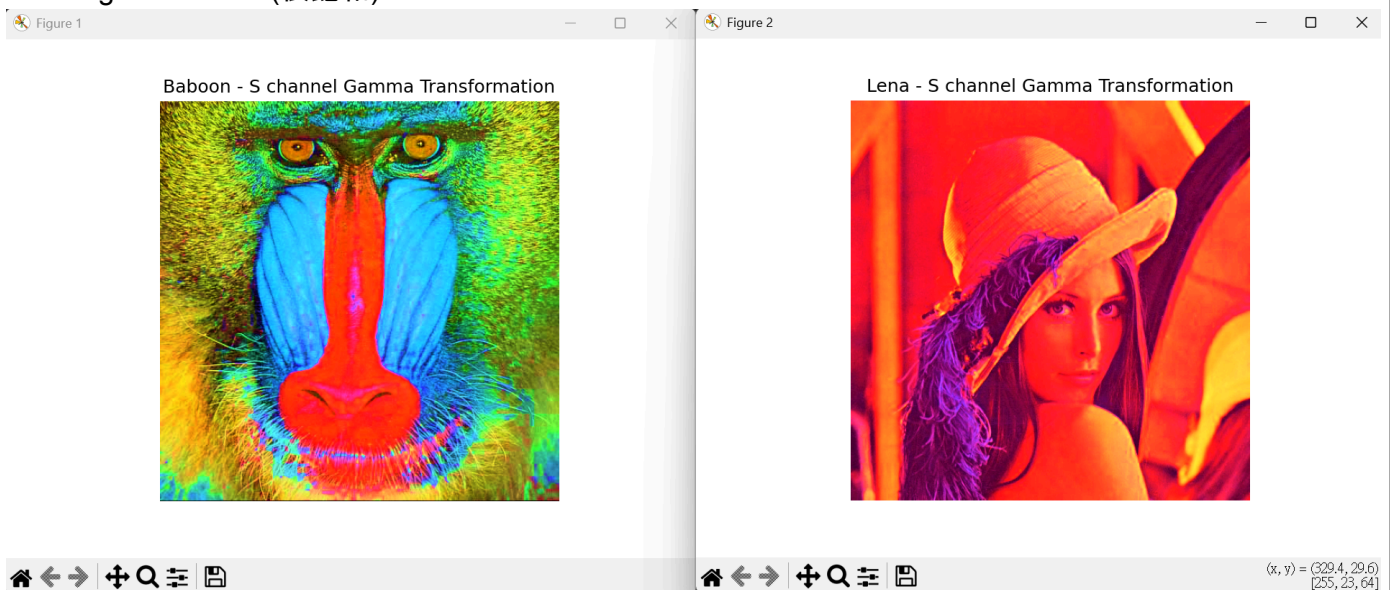
C:\Users\user\Desktop\多媒體作業二>C:/Users/user/AppData/Local/Programs/Python/Python313/python.exe c:/Users/user/Desktop/多媒體作業二/main.py
開始讀取檔案：baboon.raw
讀取成功，R, G, B 的 shape 都是：(512, 512)
開始讀取檔案：lena.raw
讀取成功，R, G, B 的 shape 都是：(512, 512)
baboon HSI 轉換完成
lena HSI 轉換完成
baboon I 通道直方圖均衡化完成！
lena I 通道直方圖均衡化完成！
baboon S 通道 Gamma 完成！
lena S 通道 Gamma 完成！

```

- gamma = 2.0 (不飽和)



- gamma = 0.2(很飽和)



- 參考資料來源：

1. Scikit-Image 官方說明_Gamma Correction:
https://scikit-image.org/docs/stable/api/skimage.exposure.html#skimage.exposure.adjust_gamma
2. 協助與偵錯:程式邏輯以及撰寫皆由本人先行完成，再使用ChatGPT協助檢查錯誤。

- 作業題目五_Step 5:以自行定義的方法進行 H Channel 的變化：

- 實作方法:為了改變影像的整體色調, 我對從 HSI 色彩空間取得的 H 通道進行自行定義的色調變化, H 通道代表顏色在色環中的角度位置(0~360度), 不同的角度對應不同的顏色, 如 0 度為偏紅色、120 度為偏綠色、240度為偏藍色, 會透過調整 H 的角度, 可以讓影像呈現偏暖、偏冷、偏紅或偏藍等不同的視覺風格。
- (1) H(Hue)代表影像的色調, 範圍為 0度~360度, 不同角度對應不同顏色。
 - (2) 為了改變整體色調, 我使用最直覺的方法:將 H 通道整體加上一個固定的角度(Hue Shift)。
 - (3) Hue Shift 會讓色環上的所有像素同時旋轉, 使影像呈現偏暖、偏冷或偏某個顏色的視覺效果。
 - (4) 色調變化的公式為: $H' = (H + \text{shift_degree}) \bmod 360$
 - (5) 使用 mod 360 是因為色調是一個環狀值, 超出範圍需包回 0~360°。
 - (6) shift_degree 的效果:+20 度偏紅橘、偏暖色、+200 度偏藍綠、偏冷色、+300 度偏紫色調
 - (7) 得到新的 H'(H_new)後, 會產生色調修改後的彩色影像。
 - (8) 程式碼撰寫 & 補充:

modify_H.py	main.py (新增 modify_H.py、轉換 rgb_to_hsi.py 的呼叫)
<pre> # Step 5:自行定義 H 通道的變化 (Hue shift) # 我選用直觀的方法:把 H 整體加上一個角度 # 例如 shift_degree = 20, 表示把色調整體往紅色方向偏移 20 度 # H 的範圍是 0~360, 所以加完後記得用 %360 包回合法範圍 import numpy as np def modify_H(H, shift_degree): # 把 H + shift_degree H_new = H + shift_degree # H 是角度, 要確保在 0~360 範圍, 所以用取模 H_new = H_new % 360 # 轉成 float32 (HSI-RGB 時需要) return H_new.astype(np.float32) </pre>	<pre> # step 1:正確讀取 RAW 彩色影像(Sequential 格式) # 檔案格式: 512x512, RGB, Sequential(Planar) # 這支程式會讀 baboon.raw 和 lena.raw # 然後把它們轉成 R、G、B 三個平面, 再顯示出來 import numpy as np import matplotlib.pyplot as plt # step 2: RGB 轉 HSI 呼叫 from rgb_to_hsi import rgb_to_hsi # step 3: 呼叫以 Histogram Equalization 進行 I channel 的加強 from hist_eq_I import hist_equalize_I # step 4: 呼叫 S 通道做 Gamma Transformation from gamma_S import gamma_transform_S # step 5: H 通道做自行定義的色調變化 from modify_H import modify_H # step 6: HSI 轉 RGB (顯示 HE、S Gamma、H channel 修改後的結果) from hsi_to_rgb import hsi_to_rgb # 影像的寬跟高(512x512) width = 512 height = 512 # 一張彩色圖要的總 byte 數 = 寬 * 高 * 3(R,G,B 三個平面) total_bytes = width * height * 3 # 然後讀一張 RAW 圖片的流程是: # 1. 用 np.fromfile 把檔案整個讀進來(uint8, 0~255) # 2. 檢查檔案大小對不對 # 3. 切成 R、G、B 三段 # 4. 每段 reshape 成 (height, width) # 5. 組回一張 RGB 圖片顯示 # 做成一個小函式, 等等 baboon 跟 lena 都會用到 def read_raw_image(filename): print(f"開始讀取檔案: {filename}") # 1. 讀檔(binary), dtype=uint8 代表每個 byte 是 0~255 data = np.fromfile(filename, dtype=np.uint8) # 2. 檢查大小是否正確 if data.size != total_bytes: print(f"檔案大小不對! 讀到 {data.size} bytes, 但正常應該是 {total_bytes} bytes") # 這邊先直接 return, 避免後面 reshape 爆掉 return None, None, None, None # 3. Sequential 排列: 所有 R 在前面, 接著所有 G, 再來是 B # 前面 width*height 個是 R # 中間 width*height 個是 G # 後面 width*height 個是 B R_flat = data[0 : width * height] G_flat = data[width * height : 2 * width * height] B_flat = data[2 * width * height : 3 * width * height] # 4. 變成 2D 陣列 (影像平面), 每個都是 512x512 </pre>

```

R = R_flat.reshape((height, width))
G = G_flat.reshape((height, width))
B = B_flat.reshape((height, width))

# 5. 把三個平面疊在一起, 變成一張 RGB 彩色圖 (高,寬,3)
img_rgb = np.dstack((R, G, B))

print(f"讀取成功, R, G, B 的 shape 都是: {R.shape}")
return R, G, B, img_rgb
# 讀取 baboon.raw
R_baboon, G_baboon, B_baboon, img_baboon =
read_raw_image("baboon.raw")

if img_baboon is not None:
    plt.figure()
    plt.imshow(img_baboon)
    plt.title("Baboon (from baboon.raw)")
    plt.axis("off") # 不顯示座標
# 讀取 lena.raw
R_lena, G_lena, B_lena, img_lena = read_raw_image("lena.raw")

if img_lena is not None:
    plt.figure()
    plt.imshow(img_lena)
    plt.title("Lena (from lena.raw)")
    plt.axis("off")
# 顯示兩張圖(如果上面都有讀成功)
plt.show()
# 到這裡為止:
# 已經拿到 R_baboon, G_baboon, B_baboon, R_lena, G_lena, B_lena
# 之後做 RGB -> HSI、HE、Gamma 都會用到它們

# step 2: RGB 轉 HSI 呼叫
# 把 baboon RAW 轉成 HSI
H_b, S_b, I_b = rgb_to_hsi(R_baboon, G_baboon, B_baboon)
print("baboon HSI 轉換完成")
# 把 lena RAW 轉成 HSI
H_l, S_l, I_l = rgb_to_hsi(R_lena, G_lena, B_lena)
print("lena HSI 轉換完成")

# step 3: 呼叫以 Histogram Equalization 進行 I channel 的加強
# 對 baboon 做 HE
# step 3-1: I 通道 HE
I_b_he = hist_equalize_I(I_b)
I_l_he = hist_equalize_I(I_l)

# step 3-2: 手動調整亮度(可調)
brightness_factor = 0.4 # >1 加亮: <1 變暗: 1 不變

# baboon
I_b_new = np.clip(I_b_he * brightness_factor, 0, 255)
print("baboon I 通道直方圖均衡化完成!")

# lena
I_l_new = np.clip(I_l_he * brightness_factor, 0, 255)
print("lena I 通道直方圖均衡化完成!")

# step 6-1: HSI 轉 RGB (顯示 HE 成果)
# 把新的 I (I_b_new) 與原本 H_b、S_b 組在一起轉回 RGB
R_b2, G_b2, B_b2 = hsi_to_rgb(H_b, S_b, I_b_new)
img_b2 = np.dstack((R_b2, G_b2, B_b2))

plt.figure()
plt.imshow(img_b2)
plt.title("Baboon - I channel Histogram Equalization")
plt.axis("off")

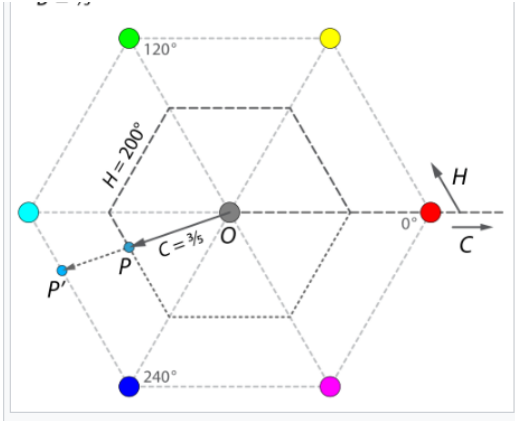
# lena 也做一次
R_l2, G_l2, B_l2 = hsi_to_rgb(H_l, S_l, I_l_new)
img_l2 = np.dstack((R_l2, G_l2, B_l2))

plt.figure()
plt.imshow(img_l2)
plt.title("Lena - I channel Histogram Equalization")
plt.axis("off")

```

	<pre>plt.show() #step 4: 呼叫 S 通道做 Gamma Transformation gamma_value = 0.25 # 你可以自己調, <1 會更鮮豔 # baboon S_b_new = gamma_transform_S(S_b, gamma_value) print("baboon S 通道 Gamma 完成!") # lena S_l_new = gamma_transform_S(S_l, gamma_value) print("lena S 通道 Gamma 完成!") # step 6-2: HSI 轉 RGB (顯示 S Gamma 的結果) # baboon. 用 S_new + I (原始 I) R_b3, G_b3, B_b3 = hsi_to_rgb(H_b, S_b_new, I_b) img_b3 = np.dstack((R_b3, G_b3, B_b3)) plt.figure() plt.imshow(img_b3) plt.title("Baboon - S channel Gamma Transformation") plt.axis("off") # lena R_l3, G_l3, B_l3 = hsi_to_rgb(H_l, S_l_new, I_l) img_l3 = np.dstack((R_l3, G_l3, B_l3)) plt.figure() plt.imshow(img_l3) plt.title("Lena - S channel Gamma Transformation") plt.axis("off") plt.show() # step 5: H 通道做自行定義的色調變化 # 設定想偏移的角度, 可自行調整 shift_deg = 180 # baboon H_b_new = modify_H(H_b, shift_deg) print("baboon H 通道色調變化完成!") # lena H_l_new = modify_H(H_l, shift_deg) print("lena H 通道色調變化完成!") # step 6-3: HSI 轉 RGB (顯示 H channel 修改後的結果) # baboon (H_new + 原始 S + 原始 I) R_b4, G_b4, B_b4 = hsi_to_rgb(H_b_new, S_b, I_b) img_b4 = np.dstack((R_b4, G_b4, B_b4)) plt.figure() plt.imshow(img_b4) plt.title("Baboon - H Channel Modified (Hue Shift)") plt.axis("off") # lena (H_new + 原始 S + 原始 I) R_l4, G_l4, B_l4 = hsi_to_rgb(H_l_new, S_l, I_l) img_l4 = np.dstack((R_l4, G_l4, B_l4)) plt.figure() plt.imshow(img_l4) plt.title("Lena - H Channel Modified (Hue Shift)") plt.axis("off") plt.show()</pre>
● 為什麼要這樣用(寫的邏輯架構): 1. 重點: H 為 0度~360度的色調角度, 調整 H 可改變影像整體色彩。 以色調位移(Hue Shift)作為自訂方法: $H_new = H + shift_degree$, 用 % 360 將角度包回合法範圍, 因為色調是環狀值。 shift_degree 會決定偏色方向: 正值偏暖/偏紅, 大角度偏藍/偏冷	補充: 有用藍底標的是新增的。

回傳 float32, 供後續 HSI 轉 RGB 使用。
下方是截自參考資料1. 的色調定義參考:

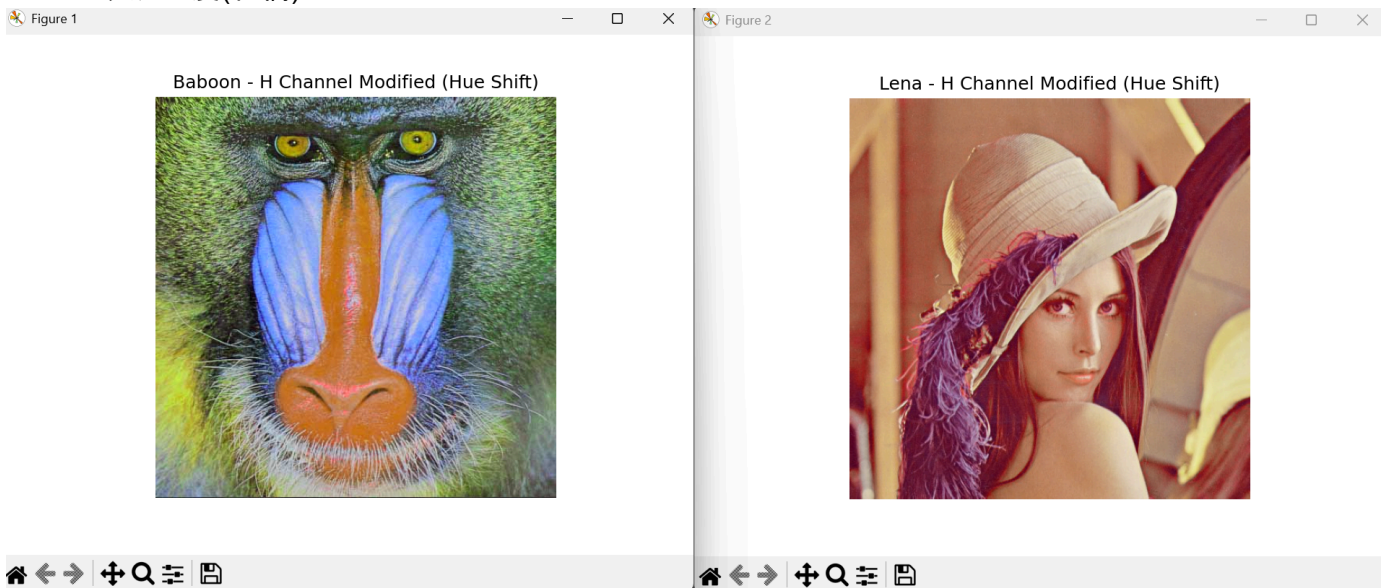


- 結果 5:

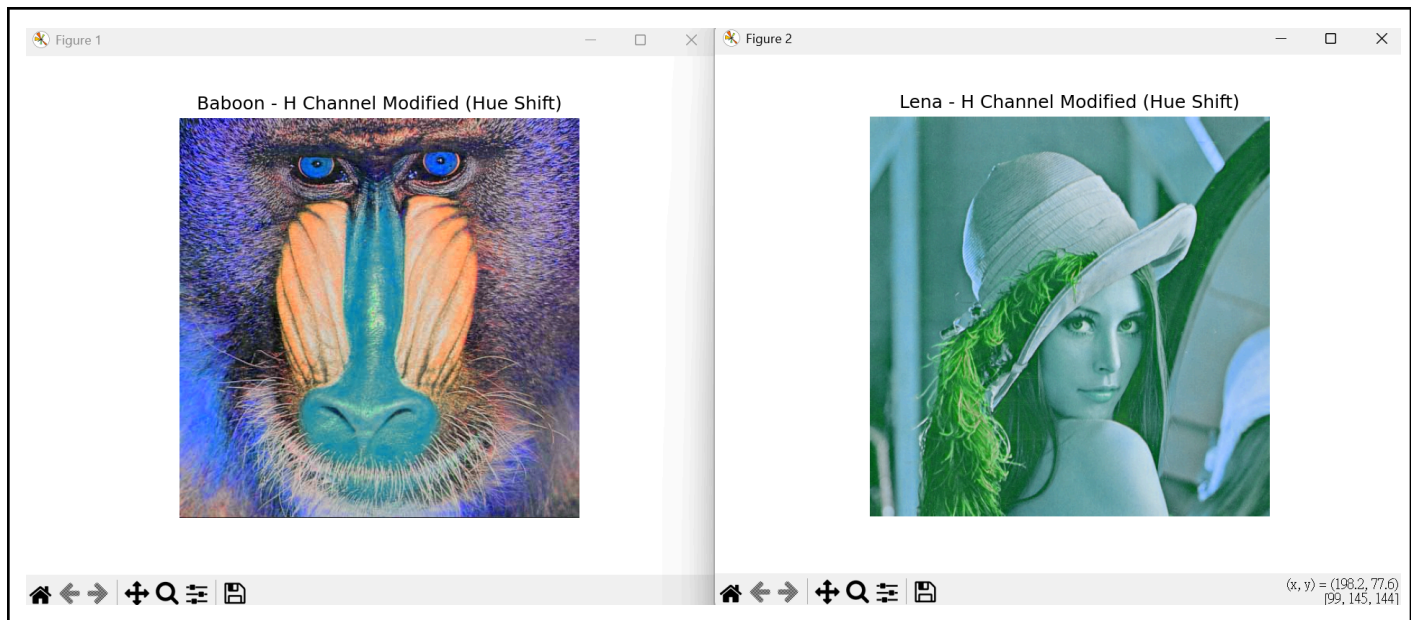
- terminal:

```
C:\Users\user\Desktop\多媒體作業二>C:/Users/user/AppData/Local/Programs/Python/Python313/python.exe c:/Users/user/Desktop/多媒體作業二/main.py
開始讀取檔案: baboon.raw
讀取成功, R, G, B 的 shape 都是: (512, 512)
開始讀取檔案: lena.raw
讀取成功, R, G, B 的 shape 都是: (512, 512)
baboon HSI 轉換完成
lena HSI 轉換完成
baboon I 通道直方圖均衡化完成!
lena I 通道直方圖均衡化完成!
baboon S 通道 Gamma 完成!
lena S 通道 Gamma 完成!
baboon H 通道色調變化完成!
lena H 通道色調變化完成!
```

- 改成20度(紅橘)



- 改成180度(綠藍)



- 參考資料:
 1. HSL and HSV(Hue 色調定義, Hue 色調是以 0~360 度表示, 所以code 用 % 360):
https://en.wikipedia.org/wiki/HSL_and_HSV
 2. 如何使用 NumPy 在 Python 中對 HSV 像素值進行偏移, 特別提到 OpenCV 中 H 值的範圍(0~179):
<https://stackoverflow.com/questions/49697363/shifting-hsv-pixel-values-in-python-using-numpy>
 3. 協助與偵錯:程式邏輯以及撰寫皆由本人先行完成, 再使用ChatGPT協助檢查錯誤。

- 作業題目六_Step 6-4最終結果(H、S、I 都做完變化後):

- (1) 程式碼撰寫 & 補充:

main.py

```
# step 1:正確讀取 RAW 彩色影像(Sequential 格式)
# 檔案格式: 512x512, RGB, Sequential(Planar)
# 這支程式會讀 baboon.raw 和 lena.raw
# 然後把它們轉成 R、G、B 三個平面, 再顯示出來
import numpy as np
import matplotlib.pyplot as plt

# step 2: RGB 轉 HSI 呼叫
from rgb_to_hsi import rgb_to_hsi

# step 3: 呼叫以 Histogram Equalization 進行 I channel 的加強
from hist_eq_I import hist_equalize_I

# step 4: 呼叫 S 通道做 Gamma Transformation
from gamma_S import gamma_transform_S

# step 5: H 通道做自行定義的色調變化
from modify_H import modify_H

# step 6: HSI 轉 RGB (顯示 HE、S Gamma、H channel 修改後的結果)
from hsi_to_rgb import hsi_to_rgb

# 影像的寬跟高(512x512)
width = 512
height = 512

# 一張彩色圖要的總 byte 數 = 寬 * 高 * 3(R,G,B 三個平面)
total_bytes = width * height * 3

# 然後讀一張 RAW 圖片的流程是:
# 1. 用 np.fromfile 把檔案整個讀進來(uint8, 0~255)
# 2. 檢查檔案大小對不對
# 3. 切成 R、G、B 三段
# 4. 每段 reshape 成 (height, width)
# 5. 組回一張 RGB 圖片顯示

# 做成一個小函式, 等等 baboon 跟 lena 都會用到
def read_raw_image(filename):
    print(f"開始讀取檔案: {filename}")
```

```

# 1. 讀檔(binary), dtype=uint8 代表每個 byte 是 0~255
data = np.fromfile(filename, dtype=np.uint8)

# 2. 檢查大小是否正確
if data.size != total_bytes:
    print(f"檔案大小不對! 讀到 {data.size} bytes, 但正常應該是 {total_bytes} bytes")
    # 這邊先直接 return, 避免後面 reshape 爆掉
    return None, None, None, None

# 3. Sequential 排列: 所有 R 在前面, 接著所有 G, 再來是 B
# 前面 width*height 個是 R
# 中間 width*height 個是 G
# 後面 width*height 個是 B
R_flat = data[0 : width * height]
G_flat = data[width * height : 2 * width * height]
B_flat = data[2 * width * height : 3 * width * height]

# 4. 變成 2D 陣列(影像平面), 每個都是 512x512
R = R_flat.reshape((height, width))
G = G_flat.reshape((height, width))
B = B_flat.reshape((height, width))

# 5. 把三個平面疊在一起, 變成一張 RGB 彩色圖 (高,寬,3)
img_rgb = np.dstack((R, G, B))

print(f"讀取成功, R, G, B 的 shape 都是: {R.shape}")
return R, G, B, img_rgb

# 讀取 baboon.raw
R_baboon, G_baboon, B_baboon, img_baboon = read_raw_image("baboon.raw")

if img_baboon is not None:
    plt.figure()
    plt.imshow(img_baboon)
    plt.title("Baboon (from baboon.raw)")
    plt.axis("off") # 不顯示座標
# 讀取 lena.raw
R_lena, G_lena, B_lena, img_lena = read_raw_image("lena.raw")

if img_lena is not None:
    plt.figure()
    plt.imshow(img_lena)
    plt.title("Lena (from lena.raw)")
    plt.axis("off")
# 顯示兩張圖(如果上面都有讀成功)
plt.show()
# 到這裡為止:
# 已經拿到 R_baboon, G_baboon, B_baboon, R_lena, G_lena, B_lena
# 之後做 RGB -> HSI, HE, Gamma 都會用到它們

# step 2: RGB 轉 HSI 呼叫
# 把 baboon RAW 轉成 HSI
H_b, S_b, I_b = rgb_to_hsi(R_baboon, G_baboon, B_baboon)
print("baboon HSI 轉換完成")
# 把 lena RAW 轉成 HSI
H_l, S_l, I_l = rgb_to_hsi(R_lena, G_lena, B_lena)
print("lena HSI 轉換完成")

# step 3: 呼叫以 Histogram Equalization 進行 I channel 的加強
# 對 baboon 做 HE
# step 3-1: I 通道 HE
I_b_he = hist_equalize_I(I_b)
I_l_he = hist_equalize_I(I_l)

# step 3-2: 手動調整亮度(可調)
brightness_factor = 0.4 # >1 加亮; <1 變暗; 1 不變

# baboon
I_b_new = np.clip(I_b_he * brightness_factor, 0, 255)
print("baboon I 通道直方圖均衡化完成!")

# lena
I_l_new = np.clip(I_l_he * brightness_factor, 0, 255)
print("lena I 通道直方圖均衡化完成!")

# step 6-1: HSI 轉 RGB (顯示 HE 成果)

```

```

# 把新的 I (I_b_new) 與原本 H_b、S_b 組在一起轉回 RGB
R_b2, G_b2, B_b2 = hsi_to_rgb(H_b, S_b, I_b_new)
img_b2 = np.dstack((R_b2, G_b2, B_b2))

plt.figure()
plt.imshow(img_b2)
plt.title("Baboon - I channel Histogram Equalization")
plt.axis("off")

# lena 也做一次
R_l2, G_l2, B_l2 = hsi_to_rgb(H_l, S_l, I_l_new)
img_l2 = np.dstack((R_l2, G_l2, B_l2))

plt.figure()
plt.imshow(img_l2)
plt.title("Lena - I channel Histogram Equalization")
plt.axis("off")
plt.show()

#step 4: 呼叫 S 通道做 Gamma Transformation
gamma_value = 0.25 # 你可以自己調, <1 會更鮮豔
# baboon
S_b_new = gamma_transform_S(S_b, gamma_value)
print("baboon S 通道 Gamma 完成!")
# lena
S_l_new = gamma_transform_S(S_l, gamma_value)
print("lena S 通道 Gamma 完成!")

# step 6-2: HSI 轉 RGB (顯示 S Gamma 的結果)
# baboon, 用 S_new + I (原始 I)
R_b3, G_b3, B_b3 = hsi_to_rgb(H_b, S_b_new, I_b)
img_b3 = np.dstack((R_b3, G_b3, B_b3))

plt.figure()
plt.imshow(img_b3)
plt.title("Baboon - S channel Gamma Transformation")
plt.axis("off")

# lena
R_l3, G_l3, B_l3 = hsi_to_rgb(H_l, S_l_new, I_l)
img_l3 = np.dstack((R_l3, G_l3, B_l3))

plt.figure()
plt.imshow(img_l3)
plt.title("Lena - S channel Gamma Transformation")
plt.axis("off")
plt.show()

# step 5: H 通道做自行定義的色調變化
# 設定想偏移的角度, 可自行調整
shift_deg = 180
# baboon
H_b_new = modify_H(H_b, shift_deg)
print("baboon H 通道色調變化完成!")
# lena
H_l_new = modify_H(H_l, shift_deg)
print("lena H 通道色調變化完成!")

# step 6-3: HSI 轉 RGB (顯示 H channel 修改後的結果)
# baboon (H_new + 原始 S + 原始 I)
R_b4, G_b4, B_b4 = hsi_to_rgb(H_b_new, S_b, I_b)
img_b4 = np.dstack((R_b4, G_b4, B_b4))

plt.figure()
plt.imshow(img_b4)
plt.title("Baboon - H Channel Modified (Hue Shift)")
plt.axis("off")

# lena (H_new + 原始 S + 原始 I)
R_l4, G_l4, B_l4 = hsi_to_rgb(H_l_new, S_l, I_l)
img_l4 = np.dstack((R_l4, G_l4, B_l4))

plt.figure()
plt.imshow(img_l4)
plt.title("Lena - H Channel Modified (Hue Shift)")

```

```

plt.axis("off")
plt.show()

# step 6-4: 最終結果 (H、S、I 都做完變化後)
# baboon 最終結果
R_b_final, G_b_final, B_b_final = hsi_to_rgb(H_b_new, S_b_new, I_b_new)
img_b_final = np.dstack((R_b_final, G_b_final, B_b_final))

plt.figure()
plt.imshow(img_b_final)
plt.title("Baboon - Final Enhanced Result")
plt.axis("off")

# lena 最終結果
R_l_final, G_l_final, B_l_final = hsi_to_rgb(H_l_new, S_l_new, I_l_new)
img_l_final = np.dstack((R_l_final, G_l_final, B_l_final))

plt.figure()
plt.imshow(img_l_final)
plt.title("Lena - Final Enhanced Result")
plt.axis("off")
plt.show()

```

補充:有用藍底標的是新增的。

- 結果:

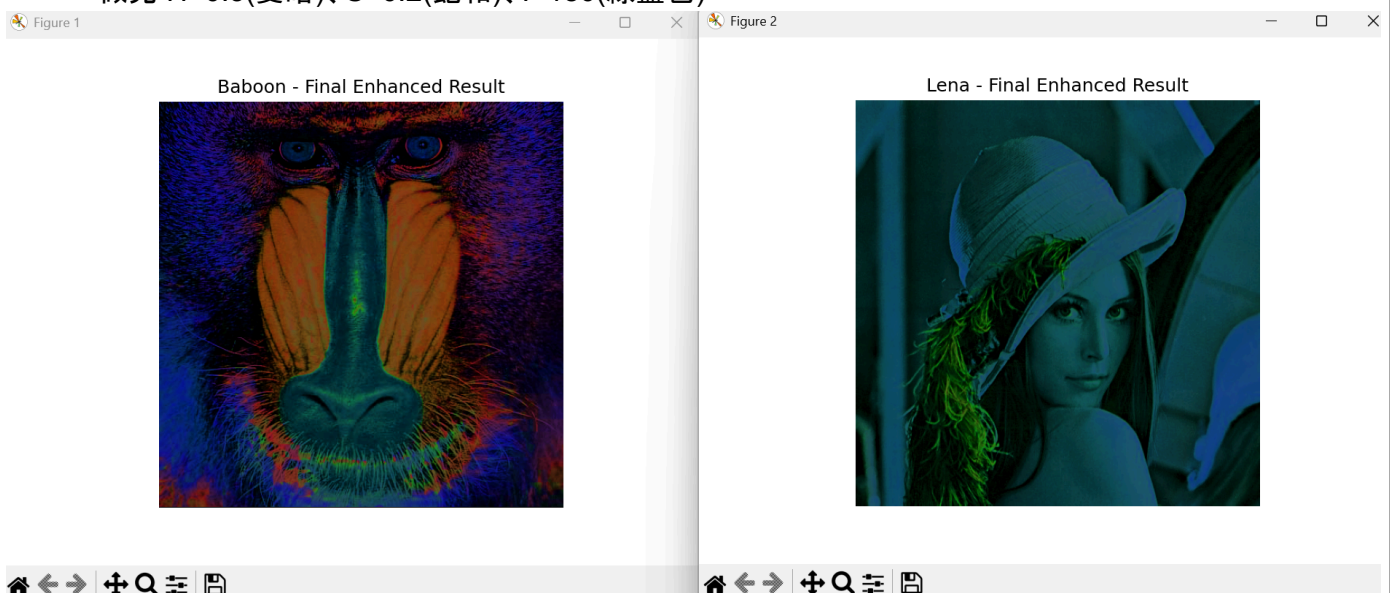
- terminal:

```

C:\Users\user\Desktop\多媒體作業二>C:/Users/user/AppData/Local/Programs/Python/Python313/python.exe c:/Users/user/Desktop/多媒體作業二/main.py
開始讀取檔案: baboon.raw
讀取成功, R, G, B 的 shape 都是: (512, 512)
開始讀取檔案: lena.raw
讀取成功, R, G, B 的 shape 都是: (512, 512)
baboon HSI 轉換完成
lena HSI 轉換完成
baboon I 通道直方圖均衡化完成!
lena I 通道直方圖均衡化完成!
baboon S 通道 Gamma 完成!
lena S 通道 Gamma 完成!
baboon H 通道色調變化完成!
lena H 通道色調變化完成!

```

- 做完 H=0.3(變暗)、S=0.2(飽和)、I=180(綠藍色)



- 作業題目七_Step 7另外加上兩張.raw檔:
- (2) 程式碼撰寫 & 補充:

main.py

```
# step 1:正確讀取 RAW 彩色影像(Sequential 格式)
# 檔案格式: 512x512, RGB, Sequential(Planar)
# 這支程式會讀 baboon.raw 和 lena.raw
# 然後把它們轉成 R、G、B 三個平面, 再顯示出來
import numpy as np
import matplotlib.pyplot as plt

# step 2: RGB 轉 HSI 呼叫
from rgb_to_hsi import rgb_to_hsi
# step 3: 呼叫以 Histogram Equalization 進行 I channel 的加強
from hist_eq_I import hist_equalize_I
#step 4: 呼叫 S 通道做 Gamma Transformation
from gamma_S import gamma_transform_S
# step 5: H 通道做自行定義的色調變化
from modify_H import modify_H
#step 6: HSI 轉 RGB (顯示 HE、 S Gamma 、H channel 修改後的結果)
from hsi_to_rgb import hsi_to_rgb

# 影像的寬跟高(512x512)
width = 512
height = 512
# 一張彩色圖要的總 byte 數 = 寬 * 高 * 3(R,G,B 三個平面)
total_bytes = width * height * 3
# 然後讀一張 RAW 圖片的流程是:
# 1. 用 np.fromfile 把檔案整個讀進來(uint8, 0~255)
# 2. 檢查檔案大小對不對
# 3. 切成 R、G、B 三段
# 4. 每段 reshape 成 (height, width)
# 5. 組回一張 RGB 圖片顯示

# 做成一個小函式, 等等 baboon 跟 lena 都會用到
def read_raw_image(filename):
    print(f"開始讀取檔案: {filename}")

    # 1. 讀檔(binary), dtype=uint8 代表每個 byte 是 0~255
    data = np.fromfile(filename, dtype=np.uint8)

    # 2. 檢查大小是否正確
    if data.size != total_bytes:
        print(f"檔案大小不對! 讀到 {data.size} bytes, 但正常應該是 {total_bytes} bytes")
        # 這邊先直接 return, 避免後面 reshape 爆掉
        return None, None, None, None

    # 3. Sequential 排列: 所有 R 在前面, 接著所有 G, 再來是 B
    # 前面 width*height 個是 R
    # 中間 width*height 個是 G
    # 後面 width*height 個是 B
    R_flat = data[0 : width * height]
    G_flat = data[width * height : 2 * width * height]
    B_flat = data[2 * width * height : 3 * width * height]

    # 4. 變成 2D 陣列(影像平面), 每個都是 512x512
    R = R_flat.reshape((height, width))
    G = G_flat.reshape((height, width))
    B = B_flat.reshape((height, width))

    # 5. 把三個平面疊在一起, 變成一張 RGB 彩色圖 (高,寬,3)
    img_rgb = np.dstack((R, G, B))

    print(f"讀取成功, R, G, B 的 shape 都是: {R.shape}")
    return R, G, B, img_rgb
# 讀取 baboon.raw
R_baboon, G_baboon, B_baboon, img_baboon = read_raw_image("baboon.raw")

if img_baboon is not None:
    plt.figure()
    plt.imshow(img_baboon)
    plt.title("Baboon (from baboon.raw)")
    plt.axis("off") # 不顯示座標
# 讀取 lena.raw
R_lena, G_lena, B_lena, img_lena = read_raw_image("lena.raw")

if img_lena is not None:
```

```

plt.figure()
plt.imshow(img_lena)
plt.title("Lena (from lena.raw)")
plt.axis("off")

# 讀取 add_1.raw
R_add1, G_add1, B_add1, img_add1 = read_raw_image("add_1.raw")

if img_add1 is not None:
    plt.figure()
    plt.imshow(img_add1)
    plt.title("add_1 (from add_1.raw)")
    plt.axis("off")

# 讀取 add_2.raw
R_add2, G_add2, B_add2, img_add2 = read_raw_image("add_2.raw")

if img_add2 is not None:
    plt.figure()
    plt.imshow(img_add2)
    plt.title("add_2 (from add_2.raw)")
    plt.axis("off")

# 顯示四張圖 (如果上面都有讀成功)
plt.show()
# 到這裡為止:
# 已經拿到 R_baboon, G_baboon, B_baboon, R_lena, G_lena, B_lena
# 之後做 RGB -> HSI, HE, Gamma 都會用到它們

# step 2: RGB 轉 HSI 呼叫
# 把 baboon RAW 轉成 HSI
H_b, S_b, I_b = rgb_to_hsi(R_baboon, G_baboon, B_baboon)
print("baboon HSI 轉換完成")
# 把 lena RAW 轉成 HSI
H_l, S_l, I_l = rgb_to_hsi(R_lena, G_lena, B_lena)
print("lena HSI 轉換完成")
# add_1 HSI
H_a1, S_a1, I_a1 = rgb_to_hsi(R_add1, G_add1, B_add1)
print("add_1 HSI 轉換完成")
# add_2 HSI
H_a2, S_a2, I_a2 = rgb_to_hsi(R_add2, G_add2, B_add2)
print("add_2 HSI 轉換完成")

# step 3: 呼叫以 Histogram Equalization 進行 I channel 的加強
# 對 baboon 做 HE
# step 3-1: I 通道 HE
I_b_he = hist_equalize_I(I_b)
I_l_he = hist_equalize_I(I_l)

# step 3-2: 手動調整亮度 (可調)
brightness_factor = 0.3 # >1 加亮; <1 變暗; 1 不變

# baboon
I_b_new = np.clip(I_b_he * brightness_factor, 0, 255)
print("baboon I 通道直方圖均衡化完成!")
# lena
I_l_new = np.clip(I_l_he * brightness_factor, 0, 255)
print("lena I 通道直方圖均衡化完成!")

# add_1
I_a1_he = hist_equalize_I(I_a1)
I_a1_new = np.clip(I_a1_he * brightness_factor, 0, 255)
print("add_1 I 通道直方圖均衡化完成!")

# add_2
I_a2_he = hist_equalize_I(I_a2)
I_a2_new = np.clip(I_a2_he * brightness_factor, 0, 255)
print("add_2 I 通道直方圖均衡化完成!")

# step 6-1: HSI 轉 RGB (顯示 HE 成果)
# 把新的 I (I_b_new) 與原本 H_b, S_b 組在一起轉回 RGB
R_b2, G_b2, B_b2 = hsi_to_rgb(H_b, S_b, I_b_new)
img_b2 = np.dstack((R_b2, G_b2, B_b2))

```

```

plt.figure()
plt.imshow(img_b2)
plt.title("Baboon - I channel Histogram Equalization")
plt.axis("off")

# lena 也做一次
R_12, G_12, B_12 = hsi_to_rgb(H_1, S_1, I_1_new)
img_12 = np.dstack((R_12, G_12, B_12))

plt.figure()
plt.imshow(img_12)
plt.title("Lena - I channel Histogram Equalization")
plt.axis("off")

# add_1: I channel Histogram Equalization
R_a1_6_1, G_a1_6_1, B_a1_6_1 = hsi_to_rgb(H_a1, S_a1, I_a1_new)
img_a1_6_1 = np.dstack((R_a1_6_1, G_a1_6_1, B_a1_6_1))

plt.figure()
plt.imshow(img_a1_6_1)
plt.title("add_1 - I channel Histogram Equalization")
plt.axis("off")

# add_2
R_a2_6_1, G_a2_6_1, B_a2_6_1 = hsi_to_rgb(H_a2, S_a2, I_a2_new)
img_a2_6_1 = np.dstack((R_a2_6_1, G_a2_6_1, B_a2_6_1))

plt.figure()
plt.imshow(img_a2_6_1)
plt.title("add_2 - I channel Histogram Equalization")
plt.axis("off")

plt.show()

#step 4: 呼叫 S 通道做 Gamma Transformation
gamma_value = 0.2 # 你可以自己調, <1 會更鮮豔
# baboon
S_b_new = gamma_transform_S(S_b, gamma_value)
print("baboon S 通道 Gamma 完成!")
# lena
S_l_new = gamma_transform_S(S_l, gamma_value)
print("lena S 通道 Gamma 完成!")

# add_1 S Gamma
S_a1_new = gamma_transform_S(S_a1, gamma_value)
print("add_1 S 通道 Gamma 完成!")

# add_2 S Gamma
S_a2_new = gamma_transform_S(S_a2, gamma_value)
print("add_2 S 通道 Gamma 完成!")

# step 6-2: HSI 轉 RGB (顯示 S Gamma 的結果)
# baboon, 用 S_new + I (原始 I)
R_b3, G_b3, B_b3 = hsi_to_rgb(H_b, S_b_new, I_b)
img_b3 = np.dstack((R_b3, G_b3, B_b3))

plt.figure()
plt.imshow(img_b3)
plt.title("Baboon - S channel Gamma Transformation")
plt.axis("off")

# lena
R_l3, G_l3, B_l3 = hsi_to_rgb(H_l, S_l_new, I_l)
img_l3 = np.dstack((R_l3, G_l3, B_l3))

plt.figure()
plt.imshow(img_l3)
plt.title("Lena - S channel Gamma Transformation")
plt.axis("off")

# add_1 - S Gamma Transformation
R_a1_6_2, G_a1_6_2, B_a1_6_2 = hsi_to_rgb(H_a1, S_a1_new, I_a1)
img_a1_6_2 = np.dstack((R_a1_6_2, G_a1_6_2, B_a1_6_2))

plt.figure()
plt.imshow(img_a1_6_2)
plt.title("add_1 - S channel Gamma Transformation")

```

```

plt.axis("off")

# add_2
R_a2_6_2, G_a2_6_2, B_a2_6_2 = hsi_to_rgb(H_a2, S_a2_new, I_a2)
img_a2_6_2 = np.dstack((R_a2_6_2, G_a2_6_2, B_a2_6_2))

plt.figure()
plt.imshow(img_a2_6_2)
plt.title("add_2 - S channel Gamma Transformation")
plt.axis("off")

plt.show()

# step 5: H 通道做自行定義的色調變化
# 設定想偏移的角度, 可自行調整
shift_deg = 90
# baboon
H_b_new = modify_H(H_b, shift_deg)
print("baboon H 通道色調變化完成!")
# lena
H_l_new = modify_H(H_l, shift_deg)
print("lena H 通道色調變化完成!")
# add_1 H shift
H_a1_new = modify_H(H_a1, shift_deg)
print("add_1 H 通道色調變化完成!")
# add_2 H shift
H_a2_new = modify_H(H_a2, shift_deg)
print("add_2 H 通道色調變化完成!")

# step 6-3: HSI 轉 RGB (顯示 H channel 修改後的結果)
# baboon (H_new + 原始 S + 原始 I)
R_b4, G_b4, B_b4 = hsi_to_rgb(H_b_new, S_b, I_b)
img_b4 = np.dstack((R_b4, G_b4, B_b4))

plt.figure()
plt.imshow(img_b4)
plt.title("Baboon - H Channel Modified (Hue Shift)")
plt.axis("off")

# lena (H_new + 原始 S + 原始 I)
R_l4, G_l4, B_l4 = hsi_to_rgb(H_l_new, S_l, I_l)
img_l4 = np.dstack((R_l4, G_l4, B_l4))

plt.figure()
plt.imshow(img_l4)
plt.title("Lena - H Channel Modified (Hue Shift)")
plt.axis("off")

# add_1 - H channel Hue Shift
R_a1_6_3, G_a1_6_3, B_a1_6_3 = hsi_to_rgb(H_a1_new, S_a1, I_a1)
img_a1_6_3 = np.dstack((R_a1_6_3, G_a1_6_3, B_a1_6_3))

plt.figure()
plt.imshow(img_a1_6_3)
plt.title("add_1 - H Channel Modified (Hue Shift)")
plt.axis("off")

# add_2
R_a2_6_3, G_a2_6_3, B_a2_6_3 = hsi_to_rgb(H_a2_new, S_a2, I_a2)
img_a2_6_3 = np.dstack((R_a2_6_3, G_a2_6_3, B_a2_6_3))

plt.figure()
plt.imshow(img_a2_6_3)
plt.title("add_2 - H Channel Modified (Hue Shift)")
plt.axis("off")

plt.show()

# step 6-4: 最終結果 (H、S、I 都做完變化後)
# baboon 最終結果
R_b_final, G_b_final, B_b_final = hsi_to_rgb(H_b_new, S_b_new, I_b_new)
img_b_final = np.dstack((R_b_final, G_b_final, B_b_final))

plt.figure()
plt.imshow(img_b_final)

```

```

plt.title("Baboon - Final Enhanced Result")
plt.axis("off")

# lena 最終結果
R_l_final, G_l_final, B_l_final = hsi_to_rgb(H_l_new, S_l_new, I_l_new)
img_l_final = np.dstack((R_l_final, G_l_final, B_l_final))

plt.figure()
plt.imshow(img_l_final)
plt.title("Lena - Final Enhanced Result")
plt.axis("off")

# add_1 最終結果
R_a1_final, G_a1_final, B_a1_final = hsi_to_rgb(H_a1_new, S_a1_new, I_a1_new)
img_a1_final = np.dstack((R_a1_final, G_a1_final, B_a1_final))

plt.figure()
plt.imshow(img_a1_final)
plt.title("add_1 - Final Enhanced Result")
plt.axis("off")

# add_2 最終結果
R_a2_final, G_a2_final, B_a2_final = hsi_to_rgb(H_a2_new, S_a2_new, I_a2_new)
img_a2_final = np.dstack((R_a2_final, G_a2_final, B_a2_final))

plt.figure()
plt.imshow(img_a2_final)
plt.title("add_2 - Final Enhanced Result")
plt.axis("off")

plt.show()

```

補充:有用藍底標的是新增的,基本上在原本baboon.raw、lena.raw有相關輸出的地方再加上add_1.raw、add_2.raw的輸出就可以了,改動沒有很多。

- 結果:

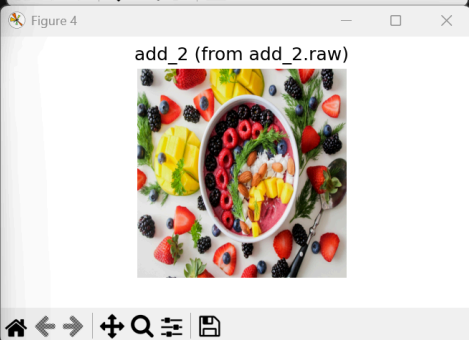
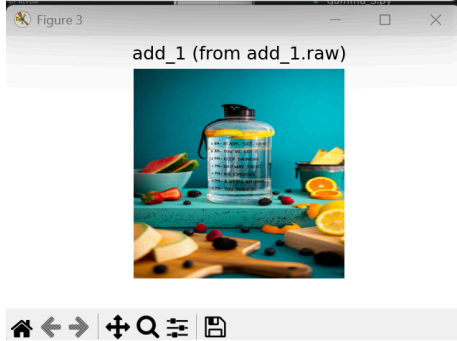
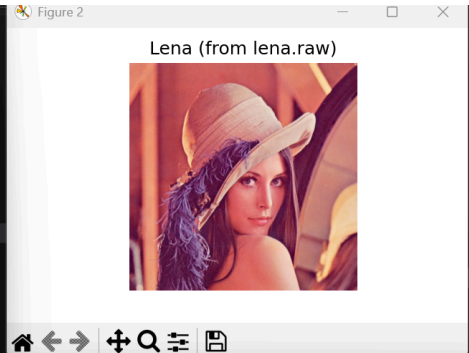
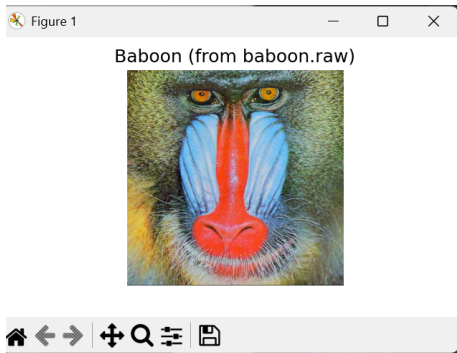
- terminal:

```

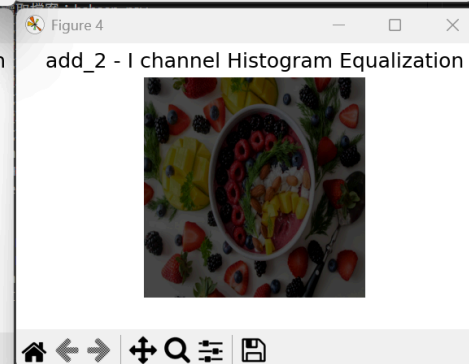
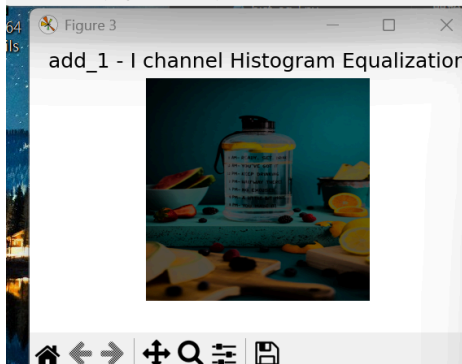
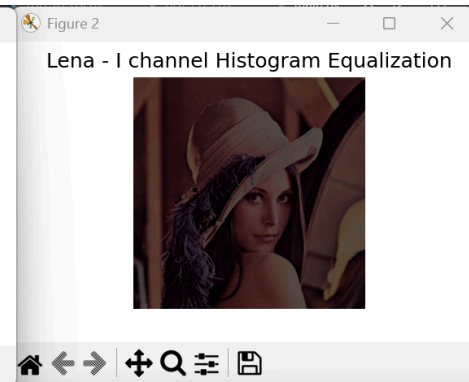
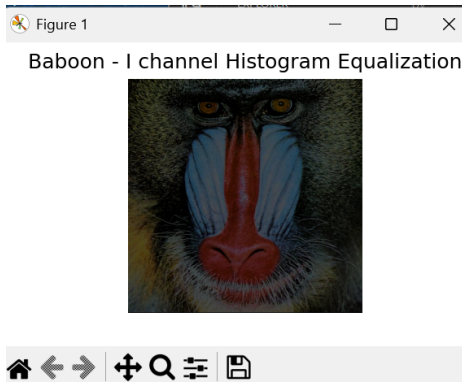
C:\Users\user\Desktop\多媒體作業二>c:/Users/user/AppData/Local/Programs/Python/Python313/python.exe c:/Users/user/Desktop/多媒體作業二/main.py
開始讀取檔案: baboon.raw
讀取成功, R, G, B 的 shape 都是: (512, 512)
開始讀取檔案: lena.raw
讀取成功, R, G, B 的 shape 都是: (512, 512)
開始讀取檔案: add_1.raw
讀取成功, R, G, B 的 shape 都是: (512, 512)
開始讀取檔案: add_2.raw
讀取成功, R, G, B 的 shape 都是: (512, 512)
lena HSI 轉換完成
c:\Users\user\Desktop\多媒體作業二\rgb_to_hsi.py:20: RuntimeWarning: invalid value encountered in divide
  S = np.where(sum_rgb == 0, 0, 1 - (3 * min_rgb / sum_rgb))
add_1 HSI 轉換完成
add_2 HSI 轉換完成
baboon I 通道直方圖均衡化完成!
lena I 通道直方圖均衡化完成!
add_1 I 通道直方圖均衡化完成!
add_2 I 通道直方圖均衡化完成!
baboon S 通道 Gamma 完成!
lena S 通道 Gamma 完成!
add_1 S 通道 Gamma 完成!
add_2 S 通道 Gamma 完成!
baboon H 通道色調變化完成!
lena H 通道色調變化完成!
add_1 H 通道色調變化完成!
add_2 H 通道色調變化完成!

```

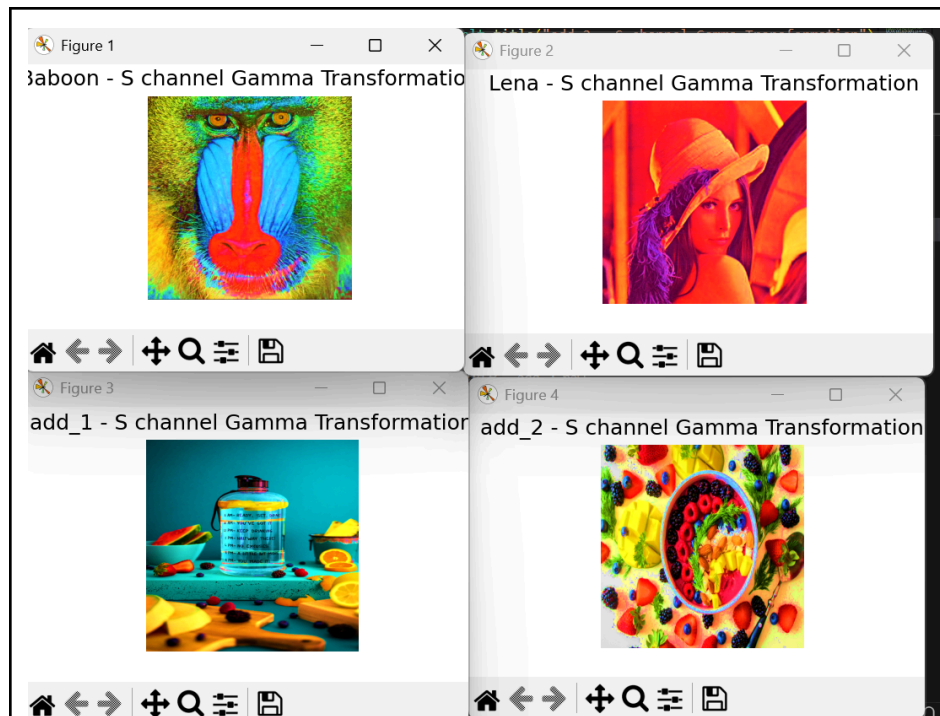
- 一開始



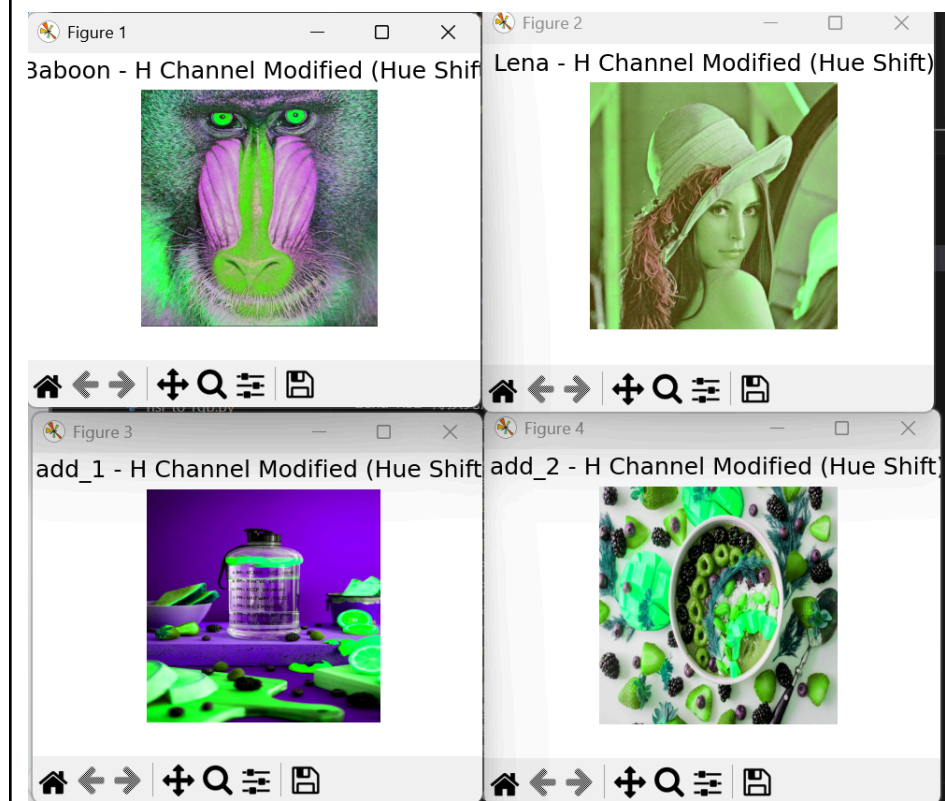
• $H = 0.3$



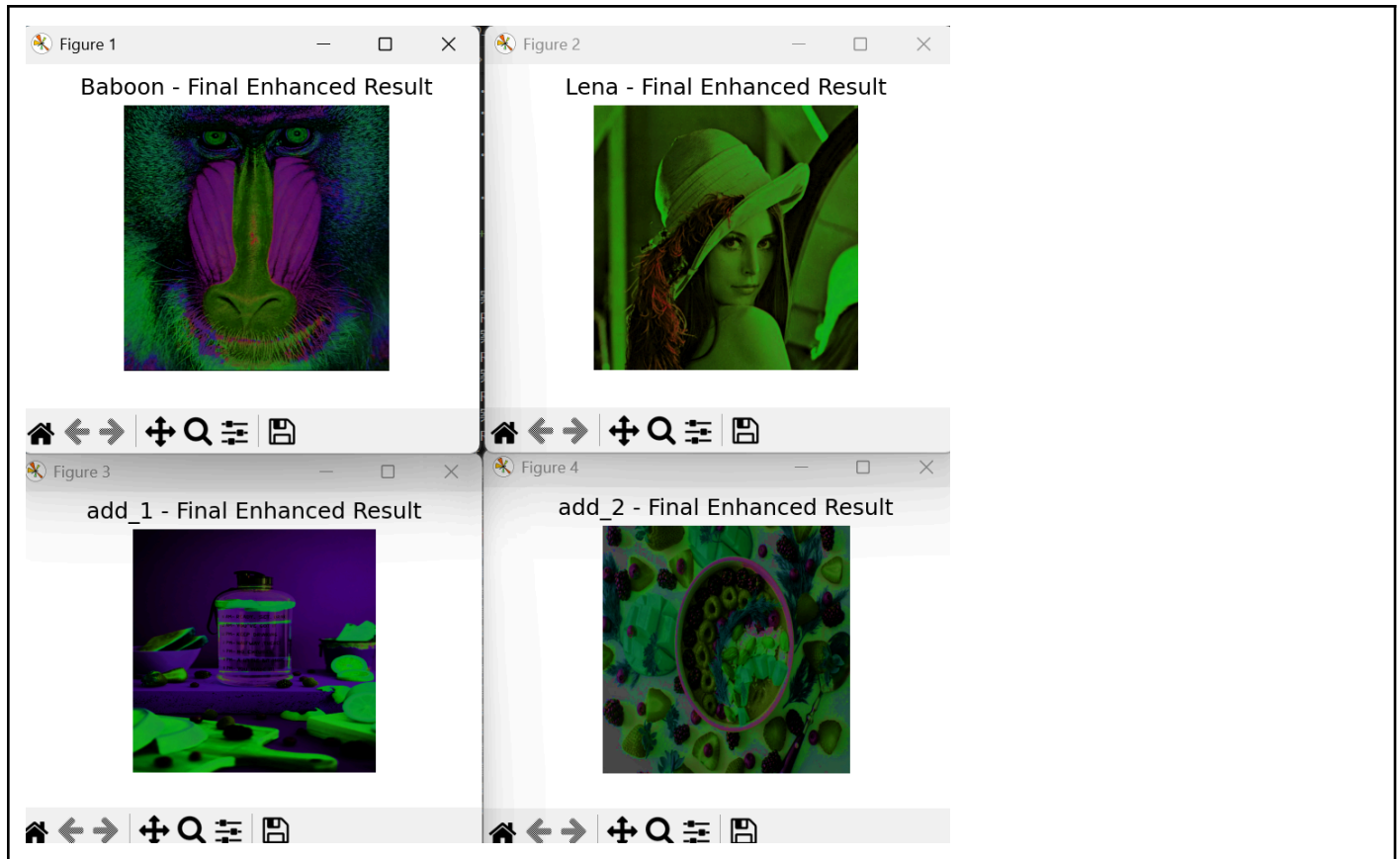
• $S = 0.2$



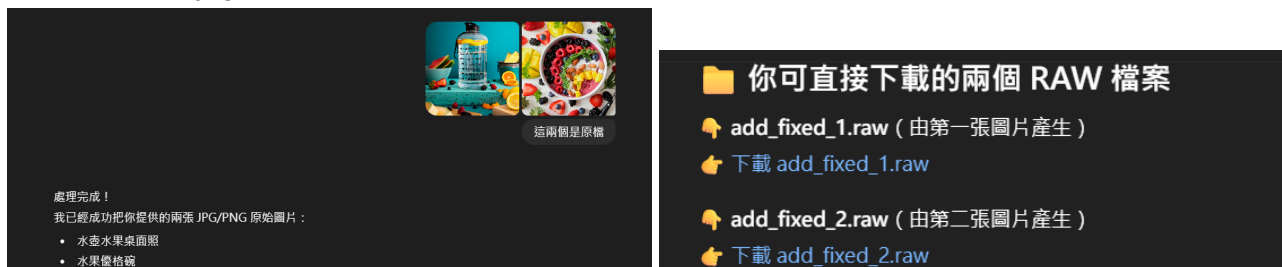
- $I = 90$



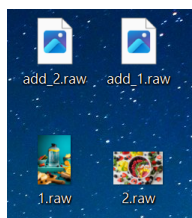
- 做完 $H=0.3$ (變暗)、 $S=0.2$ (飽和)、 $I=180$ (綠藍色)



- 參考資料來源:
- 1. 兩張照片網址:[20,000+ 張最佳 Raw 相片 · 100% 免費下載 · Pexels 圖庫相片](#)
- 2. 然後將.jpg的圖檔轉成.raw的檔案是要ChatGPT:



以上是我給他的指令對話，因為網路上的轉raw檔會轉成的檔案還是原來的.jpg不會是.raw (上面兩個.raw是ChatGPT轉成功的.raw檔，下面兩個則是我用網路上的轉.raw檔轉出來還是錯的.raw。



- 作業總心得:

透過本次作業實作，我實際從底層讀取 **RAW** 彩色影像，並在 **HSI** 色彩空間中分別對 **H**、**S**、**I** 三個通道進行影像加強，讓我更清楚理解影像資料的儲存格式以及色彩空間轉換的實際意義，相較於直接在 **RGB** 空間處理，**HSI** 能更直覺地分離亮度與色彩資訊，也使影像調整感受起來更有彈性，在實作過程中也發現參數選擇對結果影響很大，必須在效果與自然度之間取得平衡，這讓我對影像處理流程有更完整且實務面的認識。